Apprendre le framework avec documentation

Share

apj-core.zip
ZIP

APJ-docHTML-Lucas.zip
ZIP

j'ai ici une documentation d'un framework ainsi que les classes propres a ce framework, je voudrais apprendre a l'utiliser j'ai aussi un lien github d'un projet utilisant le framework si tu juges en avoir besoin pour m'aider

Ran 4 commands, read a file ›

Je suis en train de lire le guide de la doc APJ que tu m'as envoyée, il me reste les chapitres sur l'affichage, les pages et le flux complet.

Ran a command, read memory, updated memory ›

J'ai lu le guide en entier (architecture, configuration, entités, moteur de recherche, service, affichage, pages, contrôleurs `apres*`). Avec la doc et `apj-core`, j'ai ce qu'il faut. Ton lien GitHub n'est pas nécessaire pour l'instant : la doc contient un exemple complet (le module « Carton »). Il servira plus tard pour voir l'usage réel dans un projet.

On y va doucement, côté serveur Java d'abord, puis JSP, avec une validation à chaque étape et sans code.

Le chemin que je te propose

1. L'entité : une classe Java = une table (`ClassMAPTable`)
2. Les entités mère/fille et les vues de lecture
3. La recherche (`CGenUtil`)
4. Le service (`UserEJB`) : transactions, droits, workflow
5. Ensuite le côté JSP : `Page*` , `Formulaire` , `Champ` , `TableauRecherche`
6. Enfin les `apres*.jsp` : comment les données reviennent du formulaire vers Java

Étape 1 : l'entité

Dans APJ, tu n'écris pas de SQL ni de HTML à la main. Tu écris une classe Java par table, et le framework déduit tout le reste en lisant par réflexion les noms de ses attributs. C'est le principe central.

- **Chaque attribut correspond à une colonne** du même nom (la casse n'a pas d'importance). Seuls les attributs qui ont une colonne homonyme sont écrits en base. Un attribut qui n'a pas de colonne, comme un libellé calculé, n'est donc jamais inséré.

- **Les getters et setters sont obligatoires**, avec la bonne casse (`idBC` donne `getIdBC`). Le framework les appelle par réflexion, donc un accesseur mal nommé est ignoré ou provoque une erreur.
- **La classe hérite de `ClassMAPTable`** , directement ou via `ClassUser` , `ClassEtat` , `ClassMere` ou `ClassFille` . C'est elle qui fournit le CRUD, l'historique et les contrôles.
- **Dans le constructeur**, on indique la table avec `setNomTable` .
- **Deux méthodes sont à écrire.** `getTuppleID` renvoie la valeur de la clé primaire et `getAttributIDName` son nom. Un mot de vocabulaire : « tupple » veut dire « ligne de table ».
- **La clé primaire se fabrique avec `construirePK`** . Elle combine un préfixe et une séquence, ce qui donne par exemple `CRT000042` . La séquence est une fonction SQL que tu crées en base.
- **Les setters servent aussi à valider.** En mode `modif` , un setter peut lever une exception si la valeur est incorrecte, et ce message s'affiche à l'utilisateur.
- **`controler` est le hook de contrôle métier** : on y vérifie les règles avant l'insertion. Il y a aussi `controlerUpdate` et `controlerDelete` .

Deux pièges à retenir tout de suite. Utilise des types primitifs (`int` , `double`) plutôt que `Integer` / `Double` , car les wrappers nuls font planter l'insertion. Le framework met aussi en cache les colonnes de la table, donc après un `ALTER TABLE` il faut redémarrer l'application.

C'est clair jusque-là ? Si oui, je passe à l'étape 2 (entités mère/fille et vues `Lib` / `Cpl`), ou je reprends un point avant. Dis-moi aussi si tu veux qu'on regarde de vraies classes de `apj-core` en parallèle, par exemple `ClassMAPTable` .

pour valider a chaque etape, fais-moi un petit questionnaire d'environ 5 question(le niveau monte a chaque question)

Ajout à la mémoire ›

Voici ton questionnaire sur l'étape 1, 5 questions dont la difficulté monte. Réponds-y directement dans la carte. À la fin, dis-moi ce qui t'a posé problème et je reprends ces points avant de passer à l'étape 2.

on peut continuer

Reprise de la classe mère et fille, expliquée pas à pas >

Étape 2 : les documents mère/fille et les vues de lecture.

Le problème de départ

Beaucoup de documents ont une en-tête et des lignes. Par exemple, un carton (la mise en carton d'un bon de commande) a un numéro, une date et un bon de commande associé, puis plusieurs lignes : ingrédient, quantité, etc. En base, ce sont deux tables liées : `CARTON` et `CARTONFILLE`. APJ les représente par deux classes.

La classe mère (`ClassMere`)

C'est l'en-tête. Dans son constructeur, tu indiques deux choses :

- le nom de l'attribut, **situé dans la fille**, qui contient l'id de la mère (`setLiaisonFille`). Chez nous c'est `idMere`.
- le nom complet de la classe des filles (`setNomClasseFille`).

La mère sait ainsi retrouver ses lignes en base.

La classe fille (`ClassFille`)

C'est l'inverse : elle indique le nom de l'attribut qui pointe vers la mère (`setLiaisonMere`) et la classe de la mère (`setNomClasseMere`).

Piège à retenir : `nomClasseMere` est un attribut statique, donc partagé par toutes les filles. Si tu as deux types de filles différents, la dernière valeur définie écrase les autres. Il faut donc toujours la redéfinir dans le constructeur de chaque fille.

Ce que ça apporte

- **À la création**, tu donnes les lignes à la mère (`setFille`), puis tu appelles `createObject` . Le framework insère la mère, puis toutes les filles en lot, en renseignant l'id de la mère dans chaque ligne.
- **Tout est dans une seule transaction.** Si une ligne échoue (un `controler` lève une exception), tout est annulé, mère comprise.
- **À la modification et à la suppression**, une fille est refusée si la mère est déjà validée.
- **À la validation**, valider la mère valide aussi ses filles.
- **Méthodes pratiques** : relire les filles (`getFille`), dupliquer, fusionner, scinder.

Les états et l'audit (la chaîne d'héritage)

La chaîne est `ClassMAPTable` , puis `ClassUser` , puis `ClassEtat` , puis `ClassMere` / `ClassFille` .

- `ClassUser` ajoute la traçabilité : qui a créé, vérifié, rempli, et quand.
- `ClassEtat` ajoute le champ `etat` et le workflow : 0 annulé, 1 créé, 11 validé, 9 clôturé. Une fois validé (11 ou plus), on ne peut plus modifier. Dans une fiche, les boutons Modifier / Supprimer / Viser ne s'affichent donc que si l'état est inférieur à 11.

Les vues de lecture (`Lib` et `Cpl`)

L'idée est d'**écrire sur la table, mais lire sur une vue**.

- La table `CARTONFILLE` ne contient que des identifiants. À l'écran, tu veux le nom de l'ingrédient, pas son id.
- Tu crées en base une vue SQL (par exemple `CARTONFILLE_CPL`) qui fait les jointures et ajoute des colonnes comme `produitLib` .
- Tu crées une classe qui **hérite de l'entité de base**, ajoute ces attributs calculés et pointe sur la vue avec `setNomTable` . Elle s'appelle `CartonFilleCpl` , avec le suffixe `Lib` ou `Cpl` .
- Tu écris donc avec `CartonFille` et tu affiches avec `CartonFilleCpl` .
- Cela explique la question 4 du quiz : `produitLib` n'est jamais inséré, car il n'a pas de colonne dans la table d'écriture.
- Sur un attribut de vue, l'annotation `@LibelleAffichage` donne le libellé d'en-tête sans code JSP.

Un cas simple, `TypeObjet`

Pour les petits référentiels (types, catégories), `TypeObjet` offre trois colonnes toutes faites : `id` , `val` (libellé), `desce` (description). Une classe de quelques lignes suffit.

Est-ce que c'est clair ? Dis-moi si un point est flou. Sinon, je te prépare le questionnaire de 5 questions sur cette étape.

quelle est la difference entre lib et cpl

Ran a command ›
Techniquement, il n'y a **aucune différence de mécanisme** : c'est une convention de nommage, que le framework ne vérifie pas.

Ce que dit la doc

- **Lib** (« libellé ») : une vue qui remplace les identifiants par leurs noms. Par exemple, `idproduit` devient aussi `idproduitlib`, avec le nom du produit.
- **Cpl** (« complet ») : une vue ou classe enrichie de colonnes calculées ou jointes. Ça peut être des libellés, mais aussi d'autres données venant d'autres tables, comme l'id du bon de commande dans `CartonFilleCpl`.

Ce qui est identique
Dans les deux cas, on retrouve le même schéma :

- une vue SQL qui fait des jointures ;
- une classe qui hérite de l'entité de base ;
- un `setNomTable` qui pointe sur la vue ;
- une lecture sur la vue et une écriture sur la table.

Ce que le code de `apj-core` montre
Les noms peuvent aussi se combiner, par exemple `HISTORIQUE_VALEUR_LIB` ou `BC_CLIENT_FILLE_CPL_LIB_VISEE`. Cela confirme que ce sont des étiquettes que les développeurs empilent librement, pas deux types distincts.

À retenir en pratique
Quand tu crées ta propre vue d'affichage, tu choisis le suffixe selon ce qu'elle contient. Si ton équipe ou le projet a déjà sa propre règle, suis-la. Le projet GitHub que tu as mentionné pourrait nous montrer comment ils l'appliquent réellement. On peut y jeter un œil si tu veux.

Est-ce que c'est plus clair ? Si oui, je prépare le questionnaire sur l'étape 2.

lib ne contient pas des valeurs calculees?

Relecture de la documentation sur les attributs calculés ›
Oui, tu as raison de le relever : ma distinction était trop nette.

Ce que je peux affirmer

- Dans la doc, le mot « calculé » s'applique aux attributs de vue en général. Elle parle d'« attribut calculé » à propos de `produitLib`, qui n'a pas de colonne dans la table d'écriture et vient de la vue.

- Dans l'exemple de regroupement, la doc utilise `VenteLib` avec une colonne `montanttotal` qu'on somme. Cela suggère qu'une vue `Lib` peut contenir des valeurs calculées, pas seulement des noms.
- Dans le code d'`apj-core`, on voit des vues comme `BC_CLIENT_FILLE_CPL_LIB_VISEE`, où les deux suffixes sont combinés.

Ce que je ne peux pas affirmer
Le glossaire décrit `Cpl` comme « complet » (colonnes calculées ou jointes), et la doc décrit `Lib` surtout comme « jointures pour afficher les noms ». Mais **rien n'indique que `Lib` serait limité aux noms**, et le framework ne vérifie pas le suffixe : une vue SQL peut contenir ce qu'on veut.

La doc n'étant pas une source officielle (le code est décompilé), je préfère ne pas inventer une règle stricte.

Pour t'en servir

- Considère `Lib` et `Cpl` comme des étiquettes de lecture : dans les deux cas, on lit sur une vue qui enrichit la table.
- Le vrai critère, c'est la table sur laquelle pointe `setNomTable` : la table pour écrire, la vue pour lire.
- Pour connaître la règle précise de ton équipe, regarde dans le projet GitHub ce qu'ils mettent dans leurs vues `Lib` et `Cpl`. Si tu me donnes le lien, je peux le faire.

continuons d'avord

Préparation d'un quiz de cinq questions sur l'étape deux.

Voici le questionnaire de l'étape 2, 5 questions dont la difficulté monte. Il couvre les classes mère/fille, la liaison, la transaction unique, le workflow d'états et les vues de lecture. Réponds dans la carte, puis dis-moi ce qui t'a posé problème. Si tout passe, on enchaîne sur l'étape 3 : la recherche avec `CGenUtil`.

oke

Checking whether the quiz is passed.
Je pars du principe que le quiz s'est bien passé. Sinon, dis-le-moi et on reprend les points faibles.

Étape 3 : la recherche avec `CGenUtil`

Le principe : « requête par l'exemple »

Tu ne rédiges pas de `WHERE`. Tu crées un objet de l'entité (par exemple un `Carton`) et tu remplis les attributs que tu veux filtrer. Chaque attribut renseigné devient un filtre, et `CGenUtil` construit le SQL. Tu récupères un tableau d'objets du même type.

Comment chaque attribut est traité

- **Texte vide, nul ou `%`** : ignoré, donc pas de filtre.
- **Texte ordinaire** : recherche « contient », insensible à la casse (un `LIKE` avec majuscules ignorées).
- **Clé primaire** : égalité exacte.
- **Attribut déclaré avec `addAttributRecherchePrecis`** : égalité exacte aussi.
- **Nombres et dates** : jamais filtrés par l'objet. Un `int` non renseigné vaut 0 et ne signifie pas « pas de filtre ». Pour eux, on passe par les intervalles ou la clause libre.

La méthode générale `rechercher`

Elle prend cinq éléments :

1. l'objet critère ;
2. les colonnes d'intervalle ;
3. leurs valeurs ;
4. la connexion ;
5. une clause libre ajoutée à la fin, appelée `apresWhere`.

Les intervalles fonctionnent par paires : pour chaque colonne, tu donnes deux valeurs, le minimum et le maximum. C'est typique pour des dates.

La clause libre (`apresWhere`) est collée telle quelle à la requête. Elle doit commencer par `and ...` ou `order by ...`. C'est l'endroit pour les filtres sur nombres, les états ou le tri. **Ne mets jamais une saisie utilisateur brute dedans** : c'est une porte ouverte à l'injection SQL. Il faut d'abord la nettoyer avec `Utilitaire.replaceChar`.

La source des données

La requête est lue sur la table ou la vue de l'objet (`getNomTableSelect`). Si ton objet est une classe `Cpl`, la recherche se fait donc sur la vue. C'est le lien avec l'étape précédente.

Autres méthodes de la famille

Besoin	Méthode
Cas général	`rechercher`
Égalité stricte partout	`rechercherPrecis`
Mélanger AND et OR	`rechercherANDOR`
Pagination et sommes	`rechercherPage`
Regroupement (GROUP BY)	`rechercherPageGroupe`
Tableau croisé	`rechercher2D`
SQL libre	`rechercherSimple`, `findreq`

Pagination et sommes

`rechercherPage` renvoie un objet `ResultatEtSomme`. Il contient les lignes de la page demandée (le nombre par page est un paramètre), le nombre total de lignes et la somme des colonnes que tu as demandées. Avec le nombre total, `Utilitaire.calculNbPage` donne le nombre de pages.

Règle sur la connexion

Si tu passes `null`, la méthode ouvre et ferme sa propre connexion. Si tu passes une connexion, elle ne la ferme jamais. C'est important pour rester dans une transaction.

C'est clair ? Si oui, je te prépare le questionnaire de 5 questions sur cette étape. Sinon, dis-moi ce qui coince.

dans recherhce,"leurs valeurs" c'est quoi?
et dans ResultatEtSomme,la somme des colonnes que tu as demandées?

Précisant le format des valeurs par colonne ›

1. « Leurs valeurs » (les valeurs des intervalles)

Dans `rechercher`, les deuxième et troisième éléments vont ensemble :

- le deuxième est la **liste des noms de colonnes** sur lesquelles tu veux un intervalle (ce que la doc appelle `colInt`) ;
- le troisième est la **liste des valeurs** correspondantes (`valInt`).

Pour chaque colonne, il faut **deux valeurs** : le minimum puis le maximum. Les valeurs sont rangées dans le même ordre que les colonnes.

Exemple : tu veux les cartons créés en octobre 2026. Ta liste de colonnes contient `dateCreation`, et ta liste de valeurs contient `01/10/2026` puis `31/10/2026`. Le SQL généré garde les lignes dont la date est comprise entre ces deux bornes.

Avec deux colonnes d'intervalle, par exemple `dateCreation` et `montant`, la liste de valeurs a quatre éléments : min et max de la date, puis min et max du montant.

Si tu n'as pas besoin d'intervalle, tu passes `null` pour ces deux éléments, comme dans les exemples simples de la doc.

2. « La somme des colonnes que tu as demandées »

Dans `rechercherPage`, un des paramètres est une **liste de noms de colonnes numériques** que tu veux totaliser (la doc l'appelle `colSomme`). Par exemple, si tu mets `montant`, tu demandes la somme de la colonne `montant`.

Le résultat est un `ResultatEtSomme`, qui contient deux choses :

- les **lignes de la page** demandée, avec le nombre de lignes par page que tu as fixé ;
- un petit **tableau de nombres** (`getSommeEtNombre`).

D'après la doc, dans ce tableau :

- la première case est le **nombre total de lignes** correspondant à ta recherche ;
- les cases suivantes sont les **sommes**, une par colonne demandée, dans l'ordre de ta liste.

Ainsi, avec une seule colonne `montant`, tu obtiens le nombre total en première case et le total des montants en deuxième case. Si tu ne demandes aucune somme, tu n'as que le nombre.

Ces totaux semblent porter sur **toutes les lignes trouvées**, pas seulement celles de la page affichée. C'est ce qui permet de calculer le nombre de pages (avec `Utilitaire.calculNbPage`) et d'afficher un total général. C'est ma lecture de la doc, que je peux vérifier dans le code décompilé de `apj-core` si tu veux en être sûr.

Est-ce que c'est plus clair ? Si oui, je prépare le questionnaire de l'étape 3.

et avec deux colonnes d'intervalle, les parametres de rechercher augmentent alors?

Ran 2 commands ›

Non, le nombre de paramètres ne change pas. Ce sont les **tableaux** qui grandissent.

`rechercher` reçoit toujours les mêmes éléments : l'objet critère, la liste des colonnes d'intervalle, la liste des valeurs, la connexion et la clause libre. Avec deux colonnes d'intervalle, la liste des colonnes passe à 2 éléments et la liste des valeurs à 4 éléments.

J'ai vérifié l'ordre dans le code de `CGenUtil` : pour la colonne numéro i (en comptant à partir de 0), le minimum est à la position 2×i et le maximum à la position 2×i+1. Dans le code, la valeur est considérée comme absente si le tableau est trop court. Avec `dateCreation` en première et `montant` en deuxième, tu obtiens donc :

- position 0 : minimum de `dateCreation`
- position 1 : maximum de `dateCreation`
- position 2 : minimum de `montant`
- position 3 : maximum de `montant`

Si tu inverses l'ordre des colonnes, tu dois inverser les paires de valeurs aussi, sinon les bornes ne vont plus avec la bonne colonne.

Cela confirme aussi ce que je disais sur les sommes : le tableau renvoyé par `rechercherPage` fait la taille de la liste des colonnes à sommer plus un. La première case contient le nombre de lignes, et les suivantes les sommes.

Prêt pour le questionnaire de l'étape 3 ?

oui

Voici le questionnaire de l'étape 3, 5 questions dont la difficulté monte. Il couvre la requête par l'exemple, le filtre sur les textes, le cas des nombres et des dates, la position des valeurs d'intervalle et le tableau des sommes. Réponds dans la carte et dis-moi ce qui t'a posé problème. Si tout passe, on enchaîne sur l'étape 4 : le service `UserEJB` (transactions, droits, workflow).

rajoute 5 questions de niveau superieur a la 5e question

Ran a command ›

Voici 5 questions de niveau supérieur sur l'étape 3. Je n'ai pas pu les ajouter à la carte précédente, donc elles forment un deuxième quiz. Elles portent sur :

- les intervalles dont une borne est vide (j'ai vérifié le comportement dans le code de `CGenUtil`) ;
- le calcul du nombre de pages ;
- le risque d'injection SQL dans la clause libre ;
- l'effet de passer `null` comme connexion dans une transaction ;
- un cas complet qui combine tout.

Dis-moi ce qui t'a posé problème. Si tout passe, on enchaîne sur l'étape 4 : le service `UserEJB`.

continue

Checking what the guide covers and what's missing ›

Étape 4 : le service `UserEJB`

Son rôle

Les JSP n'écrivent jamais directement en base. Elles passent par un objet de service, que l'application garde en session sous le nom `u` (c'est le `UserEJB` de l'utilisateur connecté). C'est la **seule porte d'entrée** vers la base pour l'interface, et il fait trois choses :

- il gère la **transaction** (commit ou rollback) ;
- il contrôle les **droits** de l'utilisateur ;
- il applique le **workflow** (états, validation) et envoie les notifications.

Où il se trouve

`UserEJB` est une interface définie dans le projet applicatif, pas dans `apj-core`. Le framework y fait référence, donc **chaque application doit fournir sa propre implémentation**, `UserEJBBean` (environ 4 150 lignes dans l'exemple de la doc). Ici, je m'appuie donc sur la doc : le code de cette classe n'est pas dans `apj-core`.

Le schéma transactionnel

Chaque méthode publique suit le même schéma :

1. ouvrir une connexion ;
2. désactiver l'auto-commit ;
3. appeler la version « avec connexion » de la méthode ;
4. si tout va bien, faire un commit ;

5. si une exception survient, faire un rollback ;
6. fermer la connexion dans tous les cas.

La version « avec connexion » existe pour pouvoir **enchaîner plusieurs opérations dans une même transaction**. Une exception levée dans un `controler()`, un setter ou une requête annule donc tout, mère et filles comprises, et son message remonte à l'utilisateur.

Ce que fait `createObject`, étape par étape

1. **Contrôle des droits** : l'utilisateur a-t-il le droit d'insérer sur cet objet ? Sinon, exception « action de création non autorisée ».
2. **Contrôle du rang** de l'utilisateur.
3. **Préparation de l'objet** : le service se rattache à l'objet et le met en mode `modif`.
4. **Délégation à la classe `CreateObject`**, qui renseigne ce qui doit l'être : pour un objet avec état (`ClassEtat`), l'état passe à 1 et l'auteur est renseigné ; pour un `ClassUser`, l'auteur est renseigné. Puis elle appelle le `createObject` de l'entité, que tu connais déjà.

Tu retrouves ainsi, du côté service, les hooks de l'étape 1 : `controler`, puis `construirePK`, puis l'insertion avec historique.

Les droits (`ActionRole`)

`ActionRole.estAutorise` décide si l'action est permise. L'action passe si l'une de ces conditions est vraie :

- la table concernée est l'historique ;
- le rang de l'utilisateur atteint celui du directeur général ;
- la liste des règles est vide ;
- au moins une règle est satisfaite. Une règle porte sur un groupe de tables (le package), un rang minimum, une action précise (`insert`, `update`, `delete`…) ou vide pour toutes, et une direction autorisée (plusieurs possibles, ou toutes).

Le catalogue des méthodes courantes

Famille	Méthodes principales
Lecture	`getData`, `getDataPage`, `getDataPageMax`, `getDataPageGroupe`
Création	`createObject`, `createObjectMultiple` (mère + filles), `createObjectFilleMultiple`
Modification	`updateObject`, `updateObjectMultiple`
Suppression	`deleteObject`, `deleteObjectMultiple`, `deleteObjetFille`
Workflow	`validerObject`, `annulerObject`, `annulerVisa`, `cloturerObject`, `finaliser`, `rejeterObject`
Copie	`dupliquerObject`
Utilisateur	`getUser`, `isSuperUser`, `getActionRole`
Notifications	`envoyerObjectNotification`, `createNotification`, `lireNotification`

Le workflow vu du service

- **Valider** : l'état passe à 11. Il peut y avoir un visa hiérarchique avant. Sur une mère, la validation se propage aux filles.
- **Annuler** : l'état passe à 0, mais c'est refusé si l'objet est déjà validé.
- **Annuler le visa** : l'état repasse à 1, seulement si aucun supérieur n'a validé.
- **Clôturer** : l'état passe à 9.

Écrire son propre service métier

Pour une opération métier qui touche plusieurs objets, tu appliques le même schéma : ouvrir une connexion, désactiver l'auto-commit, charger tes objets avec cette connexion, appeler les méthodes du service en lui passant la même connexion, puis commit, rollback en cas d'erreur, et fermeture. Tu réutilises ainsi droits, historique et contrôles du framework au lieu de les réécrire.

Est-ce clair jusque-là ? Si oui, je prépare le questionnaire de cette étape.

Ce que fait `createObject`, étape par étape

1. Contrôle des droits : l'utilisateur a-t-il le droit d'insérer sur cet objet ? Sinon, exception « action de création non autorisée ».
2. Contrôle du rang de l'utilisateur.
3. Préparation de l'objet : le service se rattache à l'objet et le met en mode `modif`.

4. Délégation à la classe `CreateObject`, qui renseigne ce qui doit l'être : pour un objet avec état (`ClassEtat`), l'état passe à 1 et l'auteur est renseigné ; pour un `ClassUser`, l'auteur est renseigné. Puis elle appelle le `createObject` de l'entité, que tu connais déjà.

sur cette partie, le createObject mentionne au tout debut est une methode? parce que dans 4 tu mentionnes une delegation a la classe createObject et aussi chaque entite a donc son propre createObject? on doit le surdefinir dans l'entite ou c'est deja de base?

Ran 2 commands ›

Tu as raison de relever l'ambiguïté : il y a **trois choses différentes** qui portent le même nom `createObject`.

1. Le `createObject` du service (`UserEJB`)
C'est la méthode que tes JSP appellent, sur l'objet `u`. Elle contrôle les droits, ouvre la connexion et gère la transaction.

2. La classe `CreateObject` (avec un C majuscule)
C'est une classe utilitaire, du côté du projet applicatif (la doc la mentionne, elle n'est pas dans `apj-core`). Le service lui délègue un travail très précis : renseigner automatiquement l'état à 1 et l'auteur (`iduser`) avant l'insertion. Elle n'est pas dans tes entités, tu ne la touches pas.

3. Le `createObject` de l'entité
C'est une méthode de `ClassMAPTable`, donc **déjà fournie de base** à toutes tes entités. J'ai vérifié dans le code : elle enchaîne `controler`, la création de la clé primaire avec `construirePK` (seulement si l'id est vide ou « 0 »), puis l'insertion avec historique.

Alors, faut-il la surdéfinir ?
Non, en règle générale. Chaque entité hérite de cette méthode, tu n'écris rien. Ce que tu écris, ce sont les points d'extension : `controler`, `construirePK`, `getTuppleID`, etc.

La seule exception dans le framework lui-même est `ClassMere`, qui la redéfinit : elle appelle celle du parent pour insérer la mère, puis insère les filles. C'est ce qui donne le comportement mère/fille de l'étape 2.

Le fil complet quand ta JSP appelle `u.createObject(carton)`

1. le service contrôle les droits et ouvre la transaction ;
2. il délègue à la classe `CreateObject`, qui renseigne l'état et l'auteur ;
3. elle appelle la méthode `createObject` de ton entité, héritée de `ClassMAPTable` ;
4. celle-ci appelle ton `controler`, ton `construirePK`, puis fait l'insertion ;
5. le service fait le commit, ou le rollback en cas d'exception.

Est-ce que c'est plus clair ? Si oui, je prépare le questionnaire de l'étape 4.

Les droits (`ActionRole`)
`ActionRole.estAutorise` décide si l'action est permise. L'action passe si l'une de ces conditions est vraie :

- la table concernée est l'historique ;
- le rang de l'utilisateur atteint celui du directeur général ;
- la liste des règles est vide ;
- au moins une règle est satisfaite. Une règle porte sur un groupe de tables (le package), un rang minimum, une action précise (`insert` , `update` , `delete` …) ou vide pour toutes, et une direction autorisée (plusieurs possibles, ou toutes).

je ne comprends pas bien ici

Ran a command ›
Je l'avais condensé à l'excès. Je reprends avec une image simple et un exemple.

L'idée : une liste de règles, un videur à l'entrée

Chaque utilisateur a une liste de **règles de droits**. Elles sont stockées dans la table `ActionRole` et chargées avec l'utilisateur. Quand tu appelles `u.createObject(carton)` , le service demande à `ActionRole.estAutorise` si l'action est permise, en lui donnant quatre éléments : la liste des règles, l'utilisateur, l'action (`insert` , `update` , `delete` …) et l'objet concerné.

Les raccourcis qui autorisent tout de suite

Il y a trois cas où on n'a même pas besoin de regarder les règles une par une :

1. **La table visée est l'historique.** Le framework doit toujours pouvoir y écrire.
2. **Le rang de l'utilisateur est au moins celui du directeur général** (c'est ce que la doc appelle « superuser »). Son rang court-circuite tout le contrôle.
3. **La liste de règles est vide.** D'après la doc, aucune règle veut dire aucune restriction. Ça peut surprendre : l'absence de règle n'interdit rien.

Sinon, on cherche une règle qui convient

Si aucun raccourci ne s'applique, il faut qu'**au moins une règle** de la liste soit satisfaite. Une règle est satisfaite quand ses **quatre conditions** sont vraies en même temps :

- **Le groupe de tables** : la table de l'objet appartient au groupe de la règle (la doc parle de « package »).
- **Le rang minimum** : le rang de l'utilisateur est supérieur ou égal au `roleminimum` de la règle.
- **L'action** : elle correspond à l'action demandée (`insert` , `update` , `delete` …), ou la règle laisse ce champ vide, ce qui veut dire « toutes les actions ».
- **La direction** : la direction de l'utilisateur figure parmi celles que la règle autorise. Ce champ contient une liste séparée par des points-virgules, ou `%` , ou rien, ce qui veut dire

« toutes les directions ».

Un exemple

Imagine une règle : groupe « stock », rang minimum 3, action `insert`, directions « Production;Logistique ».

- Un utilisateur de rang 4 de la direction Production qui insère un objet du groupe stock : **autorisé** (les quatre conditions sont vraies).
- Un utilisateur de rang 2 : refusé par cette règle (rang trop bas).
- Un utilisateur de rang 4 de la direction Commerciale : refusé par cette règle (direction non listée).
- Le même utilisateur de rang 4 qui fait un `delete` : refusé (la règle ne couvre que `insert`).

Dans ces trois cas de refus, il faut qu'une **autre règle** de sa liste convienne, sinon l'exception « action non autorisée » est levée.

Remarque sur la source : `ActionRole` fait partie du projet applicatif, pas de `apj-core`, donc cette explication vient de la doc, que je n'ai pas pu confronter au code.

Est-ce que c'est plus clair ? Si oui, je prépare le questionnaire de l'étape 4.

et ou est-ce qu'on definit les regles?

Ran 2 commands ›

Les règles ne sont pas définies dans le code Java : ce sont des **lignes dans une table de la base de données**, la table `ActionRole`. Tu les ajoutes ou les modifies en base, pas dans une classe. Le paquet `utilisateur` de l'application fournit les classes qui les lisent (`ActionRole`, `Role`, `Action`, `Restriction`, `UserMenu`, `UtilisateurInterim`), mais `apj-core` ne contient que la partie qui les charge.

Comment les règles d'un utilisateur sont chargées

J'ai trouvé ce chargement dans `MapUtilisateur` (méthode `getActionRoles`). Pour un utilisateur donné, le framework va chercher dans `ActionRole` trois sortes de lignes :

1. **Les règles personnelles** : celles dont la colonne `utilisateur` désigne cet utilisateur.
2. **Les règles de sa direction** : celles dont la direction est la sienne, avec `utilisateur` vide.
3. **Les règles générales par rang** : celles dont `utilisateur` et `direction` sont vides et dont le `roleminimum` est inférieur ou égal à son rang.

Ces trois groupes forment la « liste de règles » que reçoit `ActionRole.estAutorise` dont je t'ai parlé. Tu peux donc donner un droit à une personne précise, à toute une direction, ou à

tous ceux qui atteignent un certain rang.

Ce que je ne sais pas

La doc ne dit pas **comment on alimente cette table** : il peut y avoir des pages d'administration dans ton projet, ou on peut insérer les lignes directement en SQL. Je ne vois pas non plus dans ce que tu m'as envoyé le détail de toutes les colonnes de la table (je ne connais avec certitude que `utilisateur`, `direction` et `roleminimum`, vus dans le code). Pour les noms exacts des colonnes et savoir comment ton équipe les remplit, le plus fiable est le script SQL de ta base ou le projet GitHub.

Est-ce plus clair ? Si oui, je prépare le questionnaire de l'étape 4.

est-ce que tu peux lire un script.dmp

Sorting out which .dmp format this file uses.

Ça dépend du format réel du fichier, car l'extension `.dmp` est utilisée pour plusieurs choses différentes.

- **Si c'est un dump texte** (par exemple un export PostgreSQL ou MySQL qui contient des `CREATE TABLE` et des `INSERT`), je peux le lire sans problème. Je pourrais alors te montrer la structure exacte de la table `ActionRole` et les règles qu'elle contient.
- **Si c'est un export binaire Oracle** (Data Pump ou l'ancien `exp`), je ne peux pas l'importer. En revanche, je peux souvent en extraire les textes lisibles, comme les définitions de tables et de vues. Les données des lignes seront plus difficiles à retrouver, et je te dirai clairement ce que je n'arrive pas à lire plutôt que de l'inventer.
- **Si le fichier est très gros**, je le parcourrai par morceaux en cherchant ce qui nous intéresse, sans tout lire.

Envoie-le-moi, comme tu as fait pour les zips, et je regarde ce qu'il contient. Si tu sais déjà quelle base de données tu utilises (Oracle, PostgreSQL…), dis-le-moi aussi : ça m'aidera à interpréter le contenu.

arbiochemstandard_20261002_160451.dmp

DMP

oracle

Ran 5 commands ›

Je lis le fichier `.dmp` : c'est un export Oracle classique dont les définitions de tables sont lisibles. J'ai trouvé que `ACTIONROLE` est vide. Je décode maintenant les rôles et leurs rangs.

Ran 2 commands ›

Je peux lire ton fichier `.dmp` : c'est un export Oracle classique (`exp`, version 11.02) du schéma `ARBIOCHEMSTANDARD`, daté du 2 octobre 2026. Les définitions de tables sont en texte lisible. Les données des lignes sont en binaire, mais je sais les décoder table par table.

La table `ACTIONROLE` est vide dans cet export.

Sa structure est la suivante : `ID`, `NOMTABLEOBJET`, `ACTION`, `ROLEMINIMUM`, `DIRECTION`, `UTILISATEUR` et `GROUPETABLE`. Ce sont les colonnes qui correspondent aux quatre conditions que je t'ai décrites. Elle contient en plus `NOMTABLEOBJET` (une table précise) et `UTILISATEUR` (une règle personnelle).

Comme il n'y a aucune ligne, la liste de règles est vide pour tous les utilisateurs. D'après la doc, **aucun contrôle de droits n'est donc appliqué par ce mécanisme** sur cette base. Je le tiens de la doc et non du code (la classe `ActionRole` n'est pas dans `apj-core`), donc c'est à confirmer sur ton application. Ton export date aussi du 2 octobre, et les règles ont pu être ajoutées depuis.

Les rôles et leurs rangs

J'ai aussi décodé la table `ROLES` : 43 rôles, avec un rang de 1 à 6. Voici quelques exemples :

Rôle	Rang
pompiste, assistants, secrétaires	1
magasins (central, transit, pièces…)	2
caisse	3
maintenancier, contremaîtres	4
gestionnaire, chefs de production, responsable vente	5
dg (directeur général), dir_usine, comptable, validateur, dsi, responsable achat	6

Un point à noter : le rôle `dg` a le rang 6, mais beaucoup d'autres rôles ont aussi 6 (comptable, validateur, magasinier central…). Or la doc dit que l'utilisateur dont le rang atteint celui du directeur général court-circuite le contrôle. Si la valeur de comparaison est bien 6, tous ces rôles contourneraient les règles, mais je ne sais pas quelle valeur le code utilise exactement. Je préfère ne pas conclure sans voir le code de `UserEJBBean`.

Si tu veux, je peux continuer à fouiller le dump (les vues, les tables `UTILISATEUR`, `USERMENU`) ou revenir au questionnaire de l'étape 4. Que préfères-tu ?

ca va, passe au questionnaire

Reviewing the interface and transaction steps for the bean ›

Voici le questionnaire de l'étape 4, 5 questions dont la difficulté monte. Il couvre :

- le rôle de `UserEJB` ;
- l'origine de `createObject` ;
- le rollback en cas d'exception ;
- un cas de droits avec `ActionRole` ;
- un service métier atomique.

Réponds dans la carte et dis-moi ce qui t'a posé problème. Si tout passe, on enchaîne sur l'étape 5 : le côté JSP (`Page*` , `Formulaire` , `Champ` , `TableauRecherche`).

je ne comprends pas tres bien , une regle est assignee a un utilisateur?

Précision du tableau et des colonnes de liste ›

Pas forcément : une règle peut viser **une personne**, **une direction** ou **tout le monde à partir d'un certain rang**. C'est la colonne `utilisateur` qui fait la différence.

Les trois types de règles

La table `ActionRole` a une colonne `utilisateur` et une colonne `direction`. D'après le code de `MapUtilisateur` (méthode `getActionRoles`), voici ce que le framework charge pour un utilisateur :

1. **Règle personnelle** : la colonne `utilisateur` contient l'identifiant de cet utilisateur. Elle ne vaut que pour lui.
2. **Règle de direction** : `utilisateur` est vide, et `direction` correspond à la direction de l'utilisateur. Elle vaut pour toute la direction.
3. **Règle générale par rang** : `utilisateur` et `direction` sont tous les deux vides, et `roleminimum` est inférieur ou égal au rang de l'utilisateur. Elle vaut pour tous ceux qui atteignent ce rang.

Ces règles, une fois rassemblées, forment la **liste de règles de l'utilisateur**. C'est cette liste que reçoit `estAutorise`, avec les quatre conditions que je t'ai décrites.

Un exemple

Pour que Rina (rang 4, direction Production) puisse faire des `insert` sur le stock, tu as plusieurs choix :

- une règle à son nom (type 1) ;
- une règle pour la direction Production (type 2) ;
- une règle générale avec un rang minimum de 4 (type 3).

Une précision sur mon exemple de tout à l'heure

Dans la question 4 du quiz, je disais « la liste contient une règle avec les directions `Production;Logistique` ». Le code de chargement compare la direction par égalité simple, alors que `estAutorise` accepte une liste séparée par des points-virgules. Je ne sais donc pas si une règle à plusieurs directions est chargée pour un utilisateur de Production. Je n'ai pas vu le code de `estAutorise` ni celui de la classe `ActionRole`, donc c'est à vérifier dans ton projet. Pour répondre à la question, prends simplement l'hypothèse de l'énoncé : cette règle est dans la liste de l'utilisateur.

Est-ce plus clair ? Si oui, tu peux répondre à la question 4 du quiz, et on enchaîne sur l'étape 5.

et le groupe c'est pour l'objet qui subira l'action?
c'est quoi la difference entre groupe et direction?

Checking what the package field is matched against ›

Oui, c'est bien ça : le **groupe** concerne l'objet qui subit l'action, et la **direction** concerne l'utilisateur qui la fait.

Le groupe (côté objet)

La table `ActionRole` a deux colonnes qui désignent l'objet visé : `NOMTABLEOBJET` (une table précise) et `GROUPETABLE` (un ensemble de tables). La doc dit que la règle porte sur un « groupe de tables (package) ». Une même règle peut donc couvrir plusieurs tables d'un coup, comme toutes celles du stock, au lieu d'écrire une ligne par table.

La direction (côté utilisateur)

C'est l'unité de l'entreprise à laquelle appartient la personne : Production, Logistique, etc. Dans le code de `MapUtilisateur`, la direction de l'utilisateur est lue dans le champ `adruser` de la table `UTILISATEUR`. Elle sert à dire « cette règle vaut pour les gens de telle direction ».

La différence en une phrase

	Groupe	Direction
Concerne	l'objet (la table touchée)	l'utilisateur qui agit
Répond à la question	« sur quoi ? »	« qui ? »

Exemple

Une règle « groupe stock, action `insert`, direction Production » veut dire : les utilisateurs de la direction Production peuvent insérer des objets des tables du groupe stock. Une personne de la direction Commerciale, même de rang plus élevé, n'est pas couverte par cette règle.

Ce que je ne sais pas

Je ne vois pas comment le code compare exactement le groupe de la règle à la table de l'objet (nom exact, préfixe, correspondance de package). La classe `ActionRole` n'est pas dans `apj-core`, et la table est vide dans ton dump, donc je n'ai aucun exemple de valeur réelle de `GROUPETABLE`. Pour le savoir, il faudrait le code de cette classe dans ton projet.

Est-ce plus clair ? Si oui, tu peux répondre au quiz de l'étape 4, puis on passe à l'étape 5.

ajoute des questions au quiz

Rédaction de la cinquième question, sur un cas combiné ›

Je n'ai pas pu ajouter ces questions à la carte précédente : elles forment donc un deuxième quiz de 5 questions, plus difficiles que les premières. Il couvre :

- les trois types de règles (personnelle, de direction, par rang) ;
- la différence entre groupe et direction dans un cas concret ;
- l'ordre exact des étapes de `createObject` ;
- le workflow d'un objet déjà validé ;
- un cas piège où deux contrôles de droits se succèdent.

Réponds dans la carte, puis dis-moi ce qui t'a posé problème. Si tout passe, on enchaîne sur l'étape 5 : le côté JSP (`Page*` , `Formulaire` , `Champ` , `TableauRecherche`).

que signifie liste utilisateur?

Ce n'est pas une liste d'utilisateurs, c'est **la liste des règles de droits d'un utilisateur donné**. J'ai employé une expression trop courte.

Ce que c'est concrètement

Quand un utilisateur se connecte, le framework charge depuis la table `ActionRole` toutes les règles qui le concernent, avec la méthode `getActionRoles` de `MapUtilisateur` . Le résultat est un tableau d'objets `ActionRole` . Il rassemble les trois sortes de règles que je t'ai décrites :

- les règles à son nom ;
- les règles de sa direction ;
- les règles générales dont le rang minimum est inférieur ou égal au sien.

C'est ce tableau qu'on appelle « la liste de règles de l'utilisateur ». Elle est propre à chaque personne : deux utilisateurs de directions et de rangs différents n'ont pas la même liste.

Comment elle sert

Quand l'utilisateur lance une action (par exemple `insert` sur un carton), le service passe cette liste à `ActionRole.estAutorise`, qui la parcourt à la recherche d'une règle qui convient.

Les deux cas particuliers

- **Liste vide** : aucune règle ne concerne l'utilisateur. D'après la doc, c'est le cas « autorisé sans restriction » (dans ton dump, la table `ActionRole` est vide, donc c'est la liste de tout le monde).
- **Liste avec une seule règle** : c'est la situation des questions du quiz. Quand j'écris « sa liste contient une seule règle », je veux dire que parmi toutes les lignes d'`ActionRole`, une seule le concerne.

Est-ce plus clair ? Si oui, tu peux reprendre les questions du quiz, et on enchaîne ensuite sur l'étape 5.

etape suivante

Ran 2 commands ›

Étape 5 : la couche d'affichage (partie 1)

Le paquet `affichage` fait deux choses : il transforme une entité en HTML, et il reconstruit l'entité depuis ce que l'utilisateur a saisi dans la requête. Comme pour le reste, tout part de la réflexion sur ta classe d'entité (étape 1).

Les quatre objets qui collaborent

- **`Page*`** est le chef d'orchestre. Il reçoit l'entité, la requête HTTP et l'utilisateur.
- **`Formulaire`** contient une liste de `Champ` et génère le HTML.
- **`Champ`** est un composant de base : un champ de saisie, une liste déroulante ou une autocomplétion.
- **`TableauRecherche`** est utilisé par `PageRecherche` pour afficher le résultat d'une recherche (tableau HTML, liens, cases à cocher).

Les principales pages

Page	Usage
`PageRecherche`	liste filtrable et paginée
`PageRechercheGroupe`	liste groupée avec sommes
`PageRechercheChoix`	pop-up de sélection
`PageInsert`	saisie simple
`PageUpdate`	modification, pré-remplie par l'id
`PageInsertMultiple` / `PageUpdateMultiple`	mère et N filles
`PageConsulte`	fiche en lecture
`PageConsulteMultiple`	fiche mère avec tableau de filles

Le `Champ`

Le framework crée les champs tout seul, et toi tu les ajustes. Pour un champ donné, tu peux changer :

- son **libellé** (le HTML est accepté) et sa **valeur par défaut** ;
- un **attribut HTML libre**, par exemple `readonly` ;
- sa **visibilité** : un champ invisible devient un `input` caché ;
- son **type** : texte, zone de texte, date, mot de passe, case à cocher, bouton radio ou éditeur riche.

Pour les champs liés à d'autres données, il y a trois mécanismes :

- **Une pop-up de choix** : on indique la page d'appel et le champ de retour.
- **Des options statiques** : deux listes parallèles, les valeurs et les libellés (par exemple 1 et 2 pour Oui et Non).
- **Une autocomplétion Ajax** : on donne la classe mappée, la colonne de valeur et la table ou vue source. Dans ce cas, le champ produit deux éléments : un libellé visible et une valeur cachée.

Il existe aussi des sous-classes spécialisées : `ChampDate` , `ChampCalcul` , `ChampAutocomplete` , `ChampCompte` , `Liste` et `ListeMultiple` .

Le `Formulaire`

La façon de remplir la liste de champs dépend de l'usage :

- **En recherche** : un champ par critère. Pour un intervalle, il crée **deux champs**, `nom1` et `nom2` , qui correspondent au minimum et au maximum de l'étape 3. Il ajoute aussi les listes de choix de colonne et d'ordre pour le tri.
- **En saisie** : `PageInsert` crée un champ par colonne de la table, sauf l'id.

- **Pour des lignes multiples** : les champs sont numérotés `nom_0`, `nom_1`, …, `nom_(N-1)`, puis regroupés.

Le piège le plus courant : avant le rendu, il faut charger en base les options de toutes les listes (`getAllData` ou `preparerDataFormu` côté page). Si tu l'oublies, tes listes déroulantes restent vides.

Le `TableauRecherche`

Tu règles les éléments suivants :

- les **en-têtes** de colonnes ;
- la **colonne cliquable** et l'URL vers laquelle elle mène ; le framework ajoute à l'URL l'identifiant de la ligne (par exemple `&id=` suivi de la valeur) ;
- un **menu contextuel** (clic droit) ;
- une **sous-ligne dépliable** qui inclut une autre page, pratique pour afficher les filles ;
- un **bouton d'action** pour le tableau à cases à cocher, et le nom de l'acte qui sera posté.

Ensuite tu choisis le rendu : simple, avec cases à cocher, avec boutons radio (sélection d'une ligne) ou en version PDF.

Remarque sur la source

Je m'appuie surtout sur la doc. Ces classes (`Champ`, `Formulaire`, `Page`…) sont dans `apj-core`, donc on peut ouvrir leur code pour vérifier un point précis si tu veux.

Est-ce que c'est clair ? Si oui, je prépare le questionnaire de cette étape. Sinon, dis-moi ce qui coince.

passe au questionnaire

Reviewing the quiz answers' link to the form fields ›

Voici le questionnaire de l'étape 5, 5 questions dont la difficulté monte. Il couvre :

- le rôle de `Page*` ;
- les champs créés par `PageInsert` ;
- la représentation d'un intervalle dans un formulaire de recherche ;
- le piège des listes déroulantes vides ;
- le réglage d'un lien cliquable dans `TableauRecherche`.

Réponds dans la carte et dis-moi ce qui t'a posé problème. Si tout passe, on enchaîne sur l'étape 6 : les pages en détail (recherche, saisie, fiche, multiple) et les contrôleurs `apres*.jsp`.

continuons

Ran 2 commands ›

Étape 6 : les pages en détail

Toutes ces pages suivent le même schéma en quatre temps : on crée la page avec une entité, on ajuste les champs, on prépare les données, puis on affiche le HTML. Je les prends l'une après l'autre.

1. `PageRecherche` : la liste filtrable

On la crée avec sept éléments :

- **L'entité** (ou la vue, étape 2) : elle détermine la table, les colonnes et les critères.
- **La requête HTTP** : on y relit les critères postés.
- **La liste des critères** : un champ par attribut proposé.
- **La liste des intervalles** : un sous-ensemble des critères, qui donnent les champs `nom1` et `nom2`.
- **Le nombre de critères par rangée** du formulaire.
- **Les colonnes affichées par défaut**, que l'utilisateur peut modifier.
- **Le nombre de sélecteurs de colonnes**, souvent égal au nombre de colonnes par défaut.

Ensuite tu règles le titre, l'utilisateur, et la page rappelée par le bouton Rechercher. Tu peux personnaliser les champs de critères, par exemple donner une valeur par défaut à `dateCreation1` et `dateCreation2`. L'appel qui fait le travail est `creerObjetPage` : il **exécute la requête** (c'est `CGenUtil` de l'étape 3, avec une liste de colonnes à sommer facultative) et construit le tableau. L'affichage comporte quatre parties : le formulaire de critères (dans un `<form>` qui poste vers la page rappelée), le tableau récapitulatif des totaux, le tableau lui-même, et la pagination.

La page reconnaît plusieurs paramètres HTTP :

- `numPag` : le numéro de page ;
- `npp` : le nombre de lignes par page (25, 50, 100 ou 200) ;
- `premier=true` : premier affichage, **aucune requête n'est exécutée**, ce qui est utile pour les listes lourdes ;
- `colonne`, `ordre` : le tri, déclenché par un clic sur un en-tête ;
- `colAffiche1..N` : les colonnes choisies ;
- `motsclesss` : la recherche plein texte sur les colonnes affichées.

Pour imposer une **clause fixe** (par exemple n'afficher que les filles d'une mère), on utilise `setAWhere`. C'est la clause libre de l'étape 3 : elle doit commencer par `and`, et toute valeur venant de l'utilisateur doit passer par `Utilitaire.replaceChar`.

2. `PageInsert` et `PageUpdate` : saisie et modification

`PageInsert` reçoit l'entité, la requête et l'utilisateur. Tu ajustes les libellés, tu appelles `preparerDataFormu` (le piège des listes vides), puis tu construis le HTML. Le formulaire poste vers une page `apres*.jsp` et contient des champs cachés qui disent quoi faire :

- `acte` : l'action (ici `insert`) ;
- `bute` : la page où rediriger après un succès ;
- `classe` : le nom complet de la classe de l'entité ;
- `nomtable` : facultatif, pour forcer la table ou la vue.

`PageUpdate` a la même forme, mais son constructeur lit l'id dans la requête et **pré-remplit** les champs, avec `acte=update`.

Rappel de l'étape 1 : l'identifiant n'est jamais saisi, parce que `construirePK` le fabrique. Si la page `apres` reçoit un paramètre au nom de la clé primaire **avec une valeur**, un `acte=insert` est automatiquement transformé en `update`.

3. `PageConsulte` : la fiche en lecture

Elle lit l'id dans l'URL (sinon elle lève l'exception « Pas d identifiant »). Pour l'affichage :

- le libellé d'un champ vient de l'annotation `@LibelleAffichage`, sinon du nom de l'attribut ;
- les dates sont au format jour/mois/année ;
- les nombres réels sont affichés avec séparateur de milliers.

C'est ici que l'état de l'étape 2 réapparaît : les boutons **Modifier, Supprimer et Viser ne s'affichent que si l'état est inférieur à 11**. Tu peux faire d'un champ un lien vers une autre fiche (par exemple le bon de commande), et il existe une alternative sans HTML manuel : des boutons et des onglets à passer à la page.

4. `PageInsertMultiple` : mère et lignes

Elle correspond à l'étape 2 (mère/fille). On la crée avec la mère, la fille, la requête, le **nombre de lignes** et l'utilisateur. Elle a **deux formulaires** : celui de la mère, et celui des filles qui produit un tableau de lignes.

- Les champs de la mère se règlent comme d'habitude.
- Les champs des filles sont **un par ligne, suffixés** `_0`, `_1`, … Pour régler un champ sur **toutes les lignes** d'un coup, on passe par le regroupement des champs multiples.
- On peut pré-remplir les lignes avec un tableau de filles (par exemple venant d'un bon de commande).

Les champs cachés à poster sont `acte`, `bute`, `classe`, **`classefille`**, **`nombreLigne`** et **`colonneMere`** (l'attribut de la fille qui reçoit l'id de la mère).

Ce que le tableau de lignes envoie :

- `nomChamp_i` : la valeur pour la ligne i (en comptant à partir de 0) ;
- `nomChamp_ilibelle` : le texte d'autocomplétion, pour l'affichage seulement ;
- `checkbox_i` : seules les **lignes cochées** sont enregistrées, les autres sont ignorées.

À retenir

Ces pages ne sauvegardent rien : elles construisent un formulaire dont les champs portent les **noms des attributs** de l'entité, et dont les champs cachés disent à la page suivante quoi faire. C'est la page `apres*.jsp` (étape 7) qui reconstruit l'objet depuis la requête et appelle `UserEJB`.

Est-ce clair ? Si oui, je prépare le questionnaire de cette étape, puis on passe à l'étape 7 (les contrôleurs `apres*.jsp` et leurs actes).

Ensuite tu règles le titre, l'utilisateur, et la page rappelée par le bouton Rechercher. Tu peux personnaliser les champs de critères, par exemple donner une valeur par défaut à `dateCreation1` et `dateCreation2`. L'appel qui fait le travail est `creerObjetPage` : il exécute la requête (c'est `CGenUtil` de l'étape 3, avec une liste de colonnes à sommer facultative) et construit le tableau. L'affichage comporte quatre parties : le formulaire de critères (dans un `<form>` qui poste vers la page rappelée), le tableau récapitulatif des totaux, le tableau lui-même, et la pagination.

Comment ca le tableau lui-meme? quel tableau?

C'est le **tableau des résultats**, celui qui liste les lignes trouvées par la recherche. Ma formulation prêtait à confusion, car il y a en fait **deux tableaux** différents.

1. Le tableau récapitulatif (les totaux)

C'est un petit tableau qui affiche les **sommes** des colonnes que tu as demandées dans `creerObjetPage` (la liste de colonnes à sommer de l'étape 3). Par exemple : « Total montant : 1 250 000 ». Si tu n'as demandé aucune somme, il n'a rien d'intéressant à montrer. On y accède par `getTableauRecap`.

2. Le tableau principal (les résultats)

C'est le tableau HTML avec une **ligne par objet trouvé** : une colonne par attribut affiché (id, bon de commande, date de création, remarque, etc.), avec les en-têtes cliquables pour trier. On y accède par `getTableau`, et c'est un `TableauRecherche` de l'étape 5. C'est lui que tu personnalises avec les libellés d'en-têtes, la colonne cliquable ou la sous-ligne dépliable.

Sur la page, du haut vers le bas

1. le **formulaire de critères** (les champs de filtre et le bouton Rechercher) ;
2. le **tableau récapitulatif** : les totaux ;
3. le **tableau principal** : les lignes de la page courante ;
4. la **pagination** : les liens de page et le choix du nombre de lignes.

Seules les lignes de la page courante sont dans le tableau principal, alors que les totaux portent sur **toutes les lignes trouvées** (comme on l'a vu avec le tableau des sommes de l'étape 3).

Est-ce plus clair ? Si oui, je prépare le questionnaire de l'étape 6.

On va reexpliquer cette etape par page comme tu l'as fait mais explique une page a la fois, et avec un quiz de 5 questions par page
recommence avec pagerecherche et detaille bien tes explications

Ran 4 commands, updated memory ›

Je viens de lire le code de `PageRecherche` dans `apj-core` pour te l'expliquer précisément. Ce que j'ai lu m'oblige à corriger deux de mes phrases plus bas.

PageRecherche : la liste filtrable, en détail

1. Le principe : une même page, deux passages

Une page de liste fonctionne en deux temps, sur le même fichier JSP :

- **Premier passage** : l'utilisateur arrive sur la page. Il n'y a pas encore de critères.
- **Passages suivants** : l'utilisateur remplit le formulaire et clique sur Rechercher. Le formulaire **poste vers la même page** (c'est le rôle de `setApres`), qui relit alors les

critères dans la requête.

À chaque passage, la page refait les mêmes étapes : lire les critères, interroger la base, construire le tableau.

2. Les paramètres du constructeur

Paramètre	Rôle
l'entité	l'objet de base : il détermine la table (ou la vue) et les colonnes
la requête HTTP	pour relire les critères postés
la liste des critères	un champ de recherche par attribut cité
la liste des intervalles	un **sous-ensemble** des critères : chacun devient deux champs, `nom1` (minimum) et `nom2` (maximum)
le nombre par rangée	combien de critères par ligne du formulaire
les colonnes affichées par défaut	les colonnes du tableau tant que l'utilisateur n'en choisit pas d'autres
le nombre de sélecteurs de colonnes	combien de listes « colonne affichée » offrir, souvent égal au nombre de colonnes par défaut

Première correction : j'ai écrit que le « nombre par rangée » règle le nombre de critères par ligne. Dans le code, la version du constructeur à sept paramètres, celle de l'exemple de la doc, passe **la valeur fixe 4** au formulaire. Cette version ignore donc ce paramètre, alors que la version à cinq paramètres l'utilise. Ne t'appuie pas dessus pour la mise en page.

3. Ce que fait le constructeur en interne

- Il garde ton entité comme **base**, et crée à part un **objet critère** : une instance neuve de la même classe. C'est sur cet objet critère que seront posées les valeurs saisies, pas sur ton entité.
- Il lit le paramètre `premier` de la requête.
- Il construit le formulaire de critères (un champ par critère, deux pour un intervalle) et les sélecteurs de colonnes.

4. Les réglages que tu fais ensuite

- `setTitre` : le titre de la page.
- `setUtilisateur` : **indispensable.** La recherche ne passe pas directement par `CGenUtil` : la page appelle les méthodes de lecture du service (`getData` ou `getDataPage` , étape 4). Elle a donc besoin de l'objet `u` de la session.
- `setLien` : le lien de base de l'application, utilisé pour construire toutes les URL.
- `setApres` : la page rappelée par le bouton Rechercher et par les liens de pagination.

- **Les champs de critères**, personnalisables par leur nom : libellé, valeur par défaut, etc. Pour un intervalle, on cible `dateCreation1` (le minimum) et `dateCreation2` (le maximum). Une valeur par défaut n'est utilisée que si l'utilisateur n'a rien saisi dans ce champ.
- `setAWhere` : une clause fixe, ajoutée à toutes les recherches (voir le point 7).
- `setNpp` : le nombre de lignes par page par défaut.

5. `creerObjetPage`, étape par étape

C'est l'appel qui fait tout le travail. Dans l'ordre :

1. il mémorise les **colonnes à sommer** (si tu en as passé) ;
2. il relit les **valeurs du formulaire** ;
3. il ouvre une connexion (si tu n'en as pas fourni), applique le **nombre de lignes par page** demandé et le **numéro de page** ;
4. il construit les critères (point 6), puis **interroge la base** ;
5. il charge en base les **options des listes déroulantes** du formulaire de critères ;
6. il prépare le **tableau récapitulatif** ;
7. il construit le **tableau principal** et la **pagination**.

Deuxième correction : à l'étape 5, la page charge elle-même les listes de ses critères. Le piège des listes vides (`preparerDataFormu`) concerne `PageInsert`, pas `PageRecherche`.

6. Comment chaque critère est interprété (la méthode `makeCritere`)

- **Un texte** : la valeur est posée sur l'objet critère, et c'est `CGenUtil` qui fait la recherche « contient » vue à l'étape 3. Si le champ est déclaré en recherche précise, c'est une égalité.
- **Un champ à choix multiple** : la page génère une condition « dans la liste » (`IN`).
- **Un intervalle** : les deux champs `nom1` et `nom2` alimentent les tableaux de colonnes et de valeurs de l'étape 3. Si l'un des champs est vide mais qu'il a une valeur par défaut, c'est elle qui sert.
- **Un nombre ou une date qui n'est pas dans la liste des intervalles** : la page ajoute dans la clause libre une condition de type `like` (ou `=` en recherche précise). Pour les dates et les nombres, le plus sûr est donc de les mettre dans la liste des intervalles.

7. Les paramètres HTTP et la clause fixe

- `premier=true` : premier affichage, **aucune requête n'est exécutée**. La page montre le formulaire et un tableau vide, ce qui est utile pour les listes lourdes.
- `numPag` : le numéro de page demandé.
- `npp` : le nombre de lignes par page. Les valeurs acceptées sont 25, 50, 100 et 200, ou la valeur par défaut réglée avec `setNpp`.
- **Le tri** : un clic sur un en-tête change la colonne et le sens, et la page ajoute un `order by` à la clause libre.
- `motsclesss` : la recherche plein texte sur les colonnes affichées par défaut.

- `setAWhere` : la clause libre de l'étape 3. Elle doit commencer par `and`, et toute valeur venue de l'utilisateur doit passer par `Utilitaire.replaceChar`.

Un cas à connaître : si tu n'indiques **aucune colonne à sommer** et que tu n'appelles pas `setNpp`, le nombre de lignes par page vaut 0. La page appelle alors `getData`, qui ramène **toutes les lignes sans pagination**. Pour une grosse table, règle `setNpp`.

8. Ce que tu affiches

- le **formulaire de critères** ;
- le **tableau récapitulatif**, une seule ligne : « Total », le nombre de lignes, puis une colonne « Somme de … » par colonne demandée ;
- le **tableau principal**, avec les lignes de la page courante ;
- la **pagination** : page précédente et suivante, avec « page X sur Y ».

Je prépare maintenant le questionnaire de cette page.

Voici le questionnaire sur `PageRecherche`, 5 questions dont la difficulté monte. Il couvre :

- l'appel qui exécute vraiment la recherche ;
- la liste des intervalles ;
- le paramètre `premier` ;
- le cas où toutes les lignes sont ramenées sans pagination ;
- la clause fixe avec `setAWhere`.

Réponds dans la carte et dis-moi ce qui t'a posé problème. Si tout passe, on passe à `PageInsert` et `PageUpdate`, toujours une page à la fois avec son quiz.

> Première correction : j'ai écrit que le « nombre par rangée » règle le nombre de critères par ligne. Dans le code, la version du constructeur à sept paramètres, celle de l'exemple de la doc, passe la valeur fixe 4 au formulaire. Cette version ignore donc ce paramètre, alors que la version à cinq paramètres l'utilise. Ne t'appuie pas dessus pour la mise en page.
>
> je n'ai pas compris

Checking which constructor version actually uses the nbRange value ›

Je reprends avec un exemple, c'est une toute petite chose.

Le contexte

Une classe Java peut avoir plusieurs constructeurs avec des paramètres différents : on dit qu'ils sont « surchargés ». `PageRecherche` en a plusieurs. Celui de l'exemple de la doc reçoit sept éléments, dont un nombre, le « nombre par rangée ». Ce nombre est censé dire combien de champs de critères on met **sur une même ligne** du formulaire. Avec 3, on s'attend à voir 3 critères côte à côte, puis la ligne suivante.

Ce que j'ai trouvé dans le code

Dans la version à sept paramètres, le constructeur reçoit bien ce nombre, mais il ne le transmet pas au formulaire. À la place, il écrit directement **4**. Que tu mettes 2, 3 ou 5, le formulaire est construit comme si tu avais mis 4.

Dans la version à cinq paramètres, en revanche, le nombre est bien transmis au formulaire et il est utilisé.

Ce que ça change pour toi

- C'est uniquement une question de **mise en page** du formulaire de critères. La recherche, les filtres et les résultats ne sont pas touchés.
- Quand tu copies l'exemple de la doc avec le chiffre 3, ne t'étonne pas de voir 4 critères par ligne : ce n'est pas une erreur de ta part.
- Je te conseille donc de ne pas compter sur ce paramètre pour contrôler l'aspect du formulaire.

Une limite de ma vérification

J'ai lu le code, mais je n'ai pas fait tourner l'application. Je ne sais donc pas si le formulaire réorganise ensuite les champs d'une autre manière, par exemple par du CSS. Si ça t'intéresse, tu peux tester dans ton projet avec 2 puis 5, et regarder si l'affichage change.

Est-ce plus clair ? Si oui, tu peux répondre au quiz de `PageRecherche`, puis on passe à `PageInsert` et `PageUpdate`.

donc je passe 5 ou 7 parametres au final?

Ran a command ›

Ni l'un ni l'autre par principe : le bon choix dépend de ce que tu veux, et le paramètre « nombre par rangée » n'est pas un bon critère. Il y a en fait **quatre** constructeurs.

Ce qui les distingue

Constructeur	Paramètres en plus	Ce qu'il apporte
5 paramètres	aucun	recherche simple, **sans** sélecteurs de colonnes
6 paramètres	colonnes par défaut	des sélecteurs de colonnes (une liste déroulante par colonne affichée)
7 paramètres	colonnes par défaut + nombre de sélecteurs	comme le précédent, avec un nombre de sélecteurs réglable (celui de la doc)
8 paramètres	les mêmes, plus un indicateur `premier`	pour forcer le premier affichage sans requête

Ce que montrent mes lectures

- Dans les versions à 5 et 6 paramètres, le « nombre par rangée » est transmis au formulaire. Seule la version à 7 paramètres écrit 4 à la place.
- Le sélecteur de colonnes est ce qui permet à l'utilisateur de choisir lui-même les colonnes du tableau. Il n'existe qu'à partir de la version à 6 paramètres.
- Avec la version à 5 paramètres, le tableau affiche toujours les colonnes que tu passes à `creerObjetPage`.

Que choisir

- Si tu veux que l'utilisateur puisse changer les colonnes affichées, prends la version à **6 ou 7 paramètres**, comme dans la doc. La mise en page des critères sera alors à 4 par rangée avec la version à 7 paramètres.
- Si tu veux une liste simple et un nombre par rangée que tu contrôles, la version à **5 paramètres** convient.

Je n'ai pas testé ces versions dans une application : je me base sur la lecture du code. Pour ton projet, le plus simple est de **suivre le constructeur utilisé par les pages existantes de ton équipe**, afin d'avoir le même comportement partout.

Est-ce que c'est clair ? Si oui, tu peux répondre au quiz de `PageRecherche`, puis on passe à `PageInsert` et `PageUpdate`.

je ne comprends pas "par range"

Ran a command ›

Une « rangée » est simplement une **ligne horizontale** du formulaire. « Nombre par rangée » veut dire : **combien de champs de critères on place côte à côte sur chaque ligne**.

Un exemple

Imagine un formulaire de recherche avec 6 critères : id, bon de commande, date min, date max, remarque, état.

- Avec **3 par rangée**, on obtient 2 lignes :
  - ligne 1 : id, bon de commande, date min ;
  - ligne 2 : date max, remarque, état.
- Avec **2 par rangée**, on obtient 3 lignes de 2 champs.
- Avec **4 par rangée**, on obtient une ligne de 4 champs et une ligne de 2.

Dans le code de `Formulaire`, le calcul est celui-là : on divise le nombre de champs par le nombre par rangée pour savoir combien de lignes il faut, puis on remplit ligne par ligne.

Pourquoi j'en ai parlé

Le constructeur de `PageRecherche` prend ce nombre en paramètre et le passe au formulaire. J'ai constaté que dans la version à 7 paramètres (celle de la doc), il met **4** quelle que soit la valeur que tu donnes. Si tu écris 3, tu auras donc 4 champs par ligne, pas 3.

Ce que ça change pour toi

C'est purement visuel : l'ordre des champs et la recherche ne changent pas. Le sujet n'est pas important pour comprendre `PageRecherche`, tu peux passer à autre chose.

Est-ce plus clair ? Si oui, tu peux répondre au quiz de `PageRecherche`, puis on passe à `PageInsert` et `PageUpdate`.

rajoute un quiz pour cette partie et continue avec la page suivante

Ran 2 commands ›

Voici d'abord le quiz sur les constructeurs de `PageRecherche` et le « nombre par rangée ».

Passons maintenant à la page suivante, `PageInsert`. J'ai lu son code dans `apj-core`. `PageUpdate` hérite de cette classe, donc je la traiterai juste après avec son propre quiz.

PageInsert : le formulaire de saisie, en détail

1. Son rôle

`PageInsert` construit un formulaire pour **créer un nouvel objet**. Elle ne sauvegarde rien : elle fabrique les champs, puis, au retour du formulaire, elle sait aussi **reconstruire l'objet** à partir de ce que l'utilisateur a saisi. La sauvegarde est faite plus loin par la page `apres*.jsp`, qui appelle `UserEJB` (étape 4).

2. Les constructeurs

Le principal prend trois éléments : l'entité, la requête et l'utilisateur. Il existe aussi une version à deux éléments (sans utilisateur). Dans celle-ci, les champs ne reçoivent pas l'utilisateur. Je n'ai pas vérifié l'effet exact, mais prends la version à trois éléments pour ne pas risquer de problème avec les listes.

3. Les trois façons de s'ouvrir, selon le paramètre `acte`

Le constructeur regarde le paramètre `acte` de l'URL :

- **absent** : un formulaire **vide**, c'est la saisie normale ;
- `update` : un formulaire **pré-rempli** avec un objet existant, dont l'id est en lecture seule ;
- `dupliquer` : pré-rempli avec un objet existant, mais **l'id est vidé**, donc l'enregistrement créera un nouvel objet.

Dans les deux derniers cas, la page lit l'id dans l'URL (le paramètre porte le nom de l'attribut clé primaire, par exemple `id`), puis charge l'objet via `UserEJB.getData`. Si rien n'est trouvé, elle lève l'exception « Pas de resultat pour votre consultation ».

4. Comment les champs sont créés (cas du formulaire vide)

La page lit la liste des colonnes de ta table (l'étape 1 : intersection des attributs et des colonnes) et crée **un champ par colonne**, sauf la clé primaire, que `construirePK` fabriquera. Le type du champ dépend du type Java :

- `int`, `double`, `float` : un `ChampCalcul` (champ numérique) ;
- `java.sql.Date` : un `ChampDate` (champ date) ;
- tout le reste : un `Champ` texte simple, vide.

Le piège du libellé : par défaut, le libellé d'un champ est le **nom de la colonne**. Il faut donc le remplacer par un libellé lisible. Le HTML est accepté, donc on peut écrire des caractères accentués avec `é`.

5. Ce que tu règles après la construction

Pour chaque champ, accessible par son nom, tu peux : changer le libellé, donner une valeur par défaut, le rendre invisible (champ caché), le mettre en lecture seule, ou lui donner une liste, une pop-up ou une autocomplétion (étape 5).

6. `preparerDataFormu` : à ne pas oublier

Cette méthode ouvre une connexion et charge en base les options de toutes les listes du formulaire. Sans elle, les listes déroulantes restent vides.

7. L'affichage

Tu construis le HTML avec `makeHtmlInsertTabIndex` (qui gère l'ordre de tabulation et les boutons), puis tu le récupères avec `getHtmlInsert`, à l'intérieur d'une balise `<form>`. Ce formulaire poste vers une page `apres*.jsp`, avec des **champs cachés** :

- `acte` : l'action à faire (ici `insert`) ;
- `bute` : la page où rediriger après un succès ;
- `classe` : le nom complet de la classe de l'entité ;
- `nomtable` : facultatif, pour forcer la table ou la vue.

On les met parce que la page `apres` ne sait pas toute seule quelle classe instancier ni quelle action faire.

8. Le chemin retour : `getObjectAvecValeur`

Quand la page `apres` reçoit le formulaire, elle utilise cette méthode pour reconstruire l'objet. Elle parcourt **tous les attributs** de la classe, héritage compris, et pour chacun cherche un paramètre de requête **du même nom**. C'est pour ça que les champs du formulaire portent le nom des colonnes. L'objet est mis en mode `modif` : ses **setters valident donc la valeur** (étape 1), et leur message d'erreur remonte à l'utilisateur. Les conversions sont :

- **texte** : si le champ est vide, elle essaie le paramètre du même nom suivi de `libelle` (le texte d'une autocomplétion) ;
- **date** : lue au format jour/mois/année (`dd/MM/yyyy`) ;
- **double** : conversion tolérante d'une chaîne en nombre ;
- **int** : un champ **vide donne 0** (pas une erreur), et une partie décimale est tronquée ;
- **nombre invalide** : l'exception « format de nombre invalide ».

9. Deux fonctions annexes

- **Fichiers joints** : la page lit trois paramètres (`fichierUploadInsert` , `...Data` , `...Name`) et les place sur l'objet.
- **Mode pop-up** : si la page est ouverte depuis le champ d'une autre page, elle ajoute des champs cachés (`champReturn` , `champUrl`) pour renvoyer l'id créé au champ appelant.

Je prépare le questionnaire de cette page.

Voici les deux quiz :

- **Constructeurs de** `PageRecherche` : 5 questions sur le nombre par rangée, ce que fait la version à 7 paramètres et le choix du constructeur.
- `PageInsert` : 5 questions sur son rôle, les champs créés, le libellé par défaut, le cas d'un `int` vide et l'ouverture en mode `dupliquer` .

Réponds dans les cartes et dis-moi ce qui t'a posé problème. Si tout passe, la page suivante est `PageUpdate`, toujours avec son explication détaillée et son quiz.

page suivante

Judging a possible failure from the code alone ›

PageUpdate : le formulaire de modification, en détail

J'ai lu son code (il est court, une centaine de lignes). C'est une sous-classe de `PageInsert` : elle en reprend tout le fonctionnement (rendu, `preparerDataFormu`, `getObjectAvecValeur`) et change seulement la façon de **construire les champs**.

1. Le rôle

`PageUpdate` construit un formulaire **déjà rempli** avec les valeurs d'un objet qui existe en base, pour que l'utilisateur les modifie. Comme `PageInsert`, elle ne sauvegarde rien : l'enregistrement se fait dans la page `apres*.jsp`, qui appelle `UserEJB`.

2. Le constructeur : trois éléments

Il reçoit l'entité, la requête et l'utilisateur. Il enchaîne quatre étapes :

1. **Il garde ton entité comme objet critère** et mémorise la requête et l'utilisateur.
2. **Il lit l'id dans l'URL.** Le paramètre porte le **nom de l'attribut clé primaire**, par exemple `?id=CRT000042`. La page le pose sur l'objet critère.
3. **Il charge l'objet existant** avec `UserEJB.getData`. Si aucune ligne ne correspond, il lève l'exception « Pas de resultat pour votre consultation ». Sinon, il prend la première ligne trouvée comme objet de base.
4. **Il construit le formulaire** avec les valeurs de cet objet.

Comme dans `PageInsert`, un paramètre `acte=dupliquer` change le comportement : le formulaire est pré-rempli, mais **l'identifiant est vidé**, et l'enregistrement créera donc un nouvel objet.

3. Comment les champs sont construits (la vraie différence avec `PageInsert`)

- **Un champ par colonne, y compris la clé primaire.** Dans `PageInsert`, la clé est exclue. Ici elle est présente, avec sa valeur, mais en **lecture seule** : l'utilisateur la voit sans pouvoir la changer.
- **Chaque champ reçoit la valeur actuelle** de l'attribut correspondant.
- **Les dates** deviennent des champs date en mode « UPDATE ».
- **Les nombres réels** sont écrits sans notation exponentielle.
- **Tous les autres champs sont des champs texte simples.** Contrairement à `PageInsert`, il n'y a pas de champ numérique spécial pour les `int` et les `double`.

- **Les libellés** valent par défaut le nom de la colonne : il faut les remplacer, comme pour `PageInsert`.

4. Pourquoi la clé primaire est présente mais en lecture seule

C'est important. Un champ en lecture seule reste **envoyé avec le formulaire**. La page `apres` reçoit donc l'identifiant, et la doc dit que, lorsque le paramètre portant le nom de la clé primaire est **non vide**, un acte `insert` devient automatiquement un `update`. Dans le cas de la duplication, c'est l'inverse : l'id est vidé, donc il n'y a pas de clé à renvoyer, et l'insertion crée un nouvel objet.

5. Les étapes d'affichage

Elles sont les mêmes que pour `PageInsert` :

1. régler les libellés et autres propriétés des champs ;
2. appeler `preparerDataFormu` pour charger les listes ;
3. construire le HTML avec `makeHtmlInsertTabIndex`, puis le récupérer avec `getHtmlInsert` dans un `<form>` ;
4. ajouter les champs cachés : `acte` (ici `update`), `bute` (la page après succès), `classe` et éventuellement `nomtable`.

Au retour du formulaire, c'est `getObjectAvecValeur` (hérité de `PageInsert`) qui reconstruit l'objet, avec les mêmes conversions que celles vues pour `PageInsert`.

6. Un risque que j'ai repéré, à vérifier chez toi

Si tu oublies de passer l'id dans l'URL, l'objet critère n'a aucune clé. D'après ma lecture du code, la recherche ne filtre alors sur rien et la page prend **la première ligne de la table**, sans erreur. Je n'ai pas testé ce cas, mais mieux vaut toujours vérifier que le lien vers la page de modification contient l'id. `PageConsulte` lève, elle, l'exception « Pas d identifiant » dans ce cas, d'après la doc.

7. Une autre chose à vérifier

Pour les champs texte, le code affiche la valeur de l'attribut avec une conversion en chaîne. Si une valeur est nulle en base, j'ai l'impression qu'elle pourrait apparaître comme le texte « null » dans le champ. Je ne l'ai pas testé : regarde sur ton projet si ça arrive, et si oui, initialise tes attributs texte à une chaîne vide dans l'entité.

Je prépare le questionnaire de cette page.

Voici le questionnaire sur `PageUpdate`, 5 questions dont la difficulté monte. Il couvre :

- la différence avec `PageInsert` ;
- l'origine de l'id ;
- le champ de la clé primaire ;
- le cas d'un id introuvable ;
- la duplication.

Réponds dans la carte et dis-moi ce qui t'a posé problème. Si tout passe, la page suivante est `PageConsulte` (la fiche en lecture), avec son explication détaillée et son quiz.

Il lit l'id dans l'URL. Le paramètre porte le nom de l'attribut clé primaire, par exemple `?id=CRT000042`. La page le pose sur l'objet critère.

de quelle page on parle?

Distinguishing the Java class from the JSP file ›

Je parlais de l'**objet `PageUpdate`** (la classe Java), pas de la page web affichée dans le navigateur. J'ai employé le mot « page » pour deux choses différentes, d'où la confusion.

Les deux sens de « page »

1. **La page web** : un fichier JSP, par exemple `carton-modif.jsp`. C'est elle que le navigateur demande et qui s'affiche. Elle a une URL.
2. **L'objet `PageUpdate`** : une classe Java que ce fichier JSP **crée** avec trois éléments : l'entité, la requête et l'utilisateur.

Comment les deux s'enchaînent

1. Sur la fiche d'un carton, l'utilisateur clique sur **Modifier**. Le lien mène vers l'URL du fichier JSP de modification, avec l'id en paramètre, par exemple `...?but=vente/miseencarton/carton-modif.jsp&id=CRT000042` (c'est ce que fait la fiche de la doc).
2. Le serveur exécute ce JSP. L'URL, avec ses paramètres, est disponible dans l'objet **requête** de cette page.
3. Dans le JSP, tu écris la création de `PageUpdate` en lui donnant cette requête.
4. Le constructeur de `PageUpdate` **lit alors le paramètre** dans la requête : l'id, qui porte le nom de l'attribut clé primaire.

Quand j'écris « Il lit l'id dans l'URL », le sujet est donc le **constructeur de `PageUpdate`**, et l'URL est celle du **JSP de modification**. L'objet ne va pas la chercher tout seul : il reçoit la requête et y lit le paramètre.

La conséquence pratique

Le lien qui mène à ton JSP de modification doit contenir cet id. C'est ce qui évite le risque que je signalais : un oubli de l'id dans le lien.

Est-ce plus clair ? Si oui, tu peux répondre au quiz de `PageUpdate`, puis on passe à `PageConsulte`.

page suivante

Ran 3 commands ›
Voici `PageConsulte`. J'ai lu son code dans `apj-core` : il contient plus de fonctions que la doc n'en décrit, et je me limite ici à l'essentiel.

PageConsulte : la fiche en lecture, en détail

1. Son rôle

`PageConsulte` affiche **tous les champs d'un objet existant, en lecture seule** : un tableau avec un libellé à gauche et la valeur à droite. Il n'y a pas de formulaire de saisie. Comme les autres pages, elle ne sauvegarde rien. En revanche, elle sert de point de départ aux actions : c'est dans la fiche que l'on place les boutons Modifier, Supprimer, Viser, etc.

2. Le constructeur : entité, requête, utilisateur

Il enchaîne ces étapes :

1. **Il vérifie l'identifiant.** Si le paramètre portant le nom de l'attribut clé primaire est **absent ou vide** dans l'URL, il lève l'exception « Pas d identifiant ».
2. Il pose cet id sur l'objet critère, puis **charge l'objet** avec `UserEJB.getData`.
3. S'il ne trouve rien, il lève « Pas de resultat pour votre consultation ».

4. Il garde la **première ligne trouvée** comme objet de base, en lui conservant la table (ou la vue) de l'objet critère.
5. Il construit les champs.

Le lien avec l'étape 2 : c'est pour cela que la doc écrit `setNomTable("carton")` avant de créer la page. On peut ainsi lire sur une **vue** (les libellés de jointure) tout en gardant la bonne table pour la fiche.

La différence avec `PageUpdate` : l'oubli de l'id est ici **protégé** par l'exception. C'est donc dans `PageUpdate` que se trouve le risque que je t'avais signalé.

3. La construction des champs (`makeChamp`)

Elle crée un champ par colonne de l'objet, avec une valeur **mise en forme pour l'affichage** :

- le **libellé** vient de l'annotation `@LibelleAffichage` de l'attribut, sinon le nom de la colonne ;
- une valeur **nulle** devient un texte vide (pas le mot « null ») ;
- une **date** est écrite au format jour/mois/année ;
- un **nombre réel** reçoit des séparateurs de milliers ;
- les apostrophes sont encodées pour ne pas casser le HTML, et les guillemets sont remplacés par un espace.

4. Ce que tu peux régler

On accède à un champ par son nom avec `getChampByName`, puis on peut :

- changer son **libellé** ;
- le **cacher** ;
- en faire un **lien** : on donne une URL de page et le nom du paramètre (par exemple `id=`), et la valeur du champ devient un lien qui y ajoute sa valeur. C'est ainsi que la fiche du carton renvoie vers la fiche du bon de commande ;
- changer l'**ordre** des champs : on donne la liste des champs à placer en premier, et les autres suivent ;
- renseigner tous les libellés d'un coup, dans l'ordre des champs ;
- définir un **titre**.

Il existe aussi un mode « rubriques » qui regroupe les champs en sections (avec une section « Autres » pour ceux qui restent), et les champs de type photo sont affichés comme des images.

5. L'affichage (`getHtml`)

- Avec **moins de 13 champs**, tout est dans un seul tableau.
- À partir de **13 champs**, la fiche est coupée en deux colonnes de taille égale.
- Les champs invisibles sont ignorés.

- Le champ `etat` d'un objet qui hérite de `ClassEtat` est affiché sous la **forme du texte de l'état** (et non le nombre). C'est le workflow de l'étape 2.
- Un champ caché `id` est ajouté, avec la valeur du **premier champ** de la liste.

6. Boutons, onglets et duplication

- **Boutons** : tu passes une liste d'objets `Button`. Le code ajoute **automatiquement** un bouton « Attacher Fichier ». Pour fabriquer son lien, il extrait le premier paramètre de l'URL en supposant que c'est `but`. Si ce n'est pas le cas, le lien sera faux : c'est fragile.
- **Onglets** : des objets `OngletOP`, qui incluent d'autres pages (typiquement les filles).
- **Duplication** : une méthode génère un petit script JavaScript qui ajoute un bouton « Dupliquer ». Ce bouton ouvre la page de modification avec `acte=dupliquer` et l'id, ce qui rejoint ce que tu as vu pour `PageInsert` et `PageUpdate`.
- **Fichiers attachés** : une méthode affiche la liste des fichiers joints à l'objet.

7. Ce que la fiche ne fait pas seule

Les boutons Modifier, Supprimer et Viser **ne disparaissent pas tout seuls** quand l'objet est validé. Dans l'exemple de la doc, c'est le JSP qui lit l'état de l'objet et n'affiche ces boutons que si l'état est inférieur à 11. Chaque bouton est un lien vers une page `apres*.jsp` avec `acte`, `bute`, `classe` et l'`id`.

Je prépare le questionnaire de cette page.

Voici le questionnaire sur `PageConsulte`, 5 questions dont la difficulté monte. Il couvre :

- le rôle de la fiche ;
- le contrôle de l'id dans le constructeur ;
- la mise en forme des valeurs ;
- le lien vers une autre fiche ;
- le masquage des boutons selon l'état.

Réponds dans la carte et dis-moi ce qui t'a posé problème. Si tout passe, la page suivante est `PageInsertMultiple` (mère et lignes), avec son explication détaillée et son quiz.

page suivante

Ran 4 commands ›

Voici `PageInsertMultiple`. J'ai lu son code dans `apj-core`, et il confirme le principe vu à l'étape 2 : une mère et plusieurs lignes saisies ensemble, enregistrées dans une seule transaction.

PageInsertMultiple : mère et lignes, en détail

1. Son rôle

Elle construit **un seul formulaire pour deux niveaux** : les champs de la **mère** (l'en-tête) et un tableau de **lignes de filles**. Comme les autres pages, elle ne sauvegarde rien : elle fabrique le formulaire, puis, au retour, reconstruit la mère et les filles. La page `apres` appelle ensuite le service pour tout enregistrer d'un coup.

2. Le constructeur

Il reçoit cinq éléments : la mère, la fille, la requête, le **nombre de lignes** à afficher et l'utilisateur. Comme pour `PageInsert`, il regarde le paramètre `acte` et choisit un des trois modes :

- **Absent : saisie vide.** Le formulaire de la mère est vide, et on affiche autant de lignes vides que le nombre demandé.
- `update` **: modification.** La page charge la mère avec l'id de l'URL (si elle n'existe pas, l'exception « Pas de resultat pour votre consultation » est levée), puis ses filles. Pour retrouver les filles, elle pose l'id de la mère sur l'attribut de liaison de la fille (`liaisonMere`, étape 2). S'il n'y a aucune fille, ce n'est pas une erreur : la liste est simplement vide. Le nombre de lignes affichées est **le nombre de filles existantes plus 10** lignes vides. L'id de la mère est renvoyé dans un champ caché.
- `dupliquer` **: copie.** Même chargement, mais l'id de la mère est vidé, ainsi que l'id et la liaison de chaque fille. Les cases des lignes existantes sont **cochées automatiquement**. Le tout sera donc enregistré comme un nouveau document.

3. Deux formulaires

- `getFormu` : le formulaire de la mère. Comme dans `PageInsert`, il contient un champ par colonne sauf la clé primaire, avec les mêmes types de champs (numérique, date, texte).
- `getFormufle` : le formulaire des filles. Pour **chaque ligne** (de 0 à N-1) et **chaque colonne** de la fille, un champ est créé. Son nom est celui de la colonne suivi du numéro de ligne, avec un tiret bas : par exemple `quantite_0`, `quantite_1`, etc.

Les champs de la fille incluent **toutes** les colonnes, y compris l'id et la colonne de liaison. C'est pourquoi l'exemple de la doc **cache** ces champs, pour toutes les lignes d'un coup, en passant par le regroupement des champs de même nom (`getChampMulitple`).

4. Pré-remplir les lignes : `setDefautFille`

On lui donne un tableau de filles (par exemple celles construites à partir d'un bon de commande). La méthode :

- **change le nombre de lignes** : pour un tableau de moins de 10 filles, il y a autant de lignes que de filles plus 10 ; à partir de 10, il y en a le nombre de filles plus 3. Cela **remplace** le nombre que tu as passé au constructeur ;
- **coche automatiquement** les lignes qu'elle remplit ;
- **remplit les valeurs** de chaque ligne (les dates mises en forme, les réels avec séparateurs de milliers).

5. Les listes : `preparerDataFormu`

Comme pour `PageInsert`, il faut l'appeler avant le rendu. Ici, elle charge les options des listes de la mère **et** celles de toutes les lignes de filles.

6. L'affichage et les champs cachés

On construit le HTML des deux formulaires, puis on les affiche l'un après l'autre dans un même `<form>`. Ce formulaire poste vers une page `apres*.jsp` avec des champs cachés :

- `acte` : l'action (ici `insert`) ;
- `bute` : la page de destination après succès ;
- `classe` et `classefille` : les noms complets des classes de la mère et de la fille ;
- `nombreLigne` : le nombre de lignes affichées ;
- `colonneMere` : l'attribut de la fille qui doit recevoir l'id de la mère.

7. Le chemin retour

Au retour, plusieurs méthodes reconstruisent les objets :

- `getObjectAvecValeur` : reconstruit la mère, avec les mêmes conversions que pour `PageInsert` (date, `int` vide égal 0, etc.), mais sans toucher à l'attribut de liaison.
- `getObjectFilleAvecValeur` : reconstruit **uniquement les filles cochées**. Pour chaque ligne cochée, elle lit les paramètres `nom_i`. Les lignes non cochées sont **ignorées**. Si

aucune ligne n'est cochée, elle lève l'exception « Vous devez cocher ».

- `getObjectFilleAvecValeurSansControle` : la même chose, mais, si rien n'est coché, elle renvoie une liste vide au lieu de lever une exception.

La liste des numéros de lignes cochées est fournie au constructeur de la page, dans la page `apres`. D'après la doc, elle vient du paramètre `ids` de la requête.

Je prépare le questionnaire de cette page.

Voici le questionnaire sur `PageInsertMultiple`, 5 questions dont la difficulté monte. Il couvre :

- le nommage des champs de lignes ;
- le rôle des cases cochées ;
- le calcul du nombre de lignes avec `setDefautFille` ;
- le mode `dupliquer` ;
- le cas où aucune ligne n'est cochée.

Réponds dans la carte et dis-moi ce qui t'a posé problème. Si tout passe, la page suivante est `PageConsulteMultiple` (la fiche d'une mère avec son tableau de filles), avec son explication détaillée et son quiz.

est-ce que on peut avoir le script de la base de donnees a partir du .dmp que je t'ai envoye?

Deciding to create a .sql file ›
Oui, c'est possible pour la structure (tables, vues, séquences, index, contraintes). Je commence par repérer ce que le fichier contient.

Ran 4 commands ›
Je vois la structure complète : le dump contient 516 tables, 1 347 séquences, 1 309 vues, 424 fonctions, 8 procédures et des triggers. Je génère maintenant un script SQL propre et ordonné.

Ran a command ›
Les décomptes correspondent exactement à ceux du dump. Je vérifie maintenant qu'aucun cas particulier ne m'a échappé et que le fichier est propre.

Ran 9 commands, shared files, and 2 more tools ›
Oui, j'ai pu extraire la structure de la base : le script est dans `arbiochemstandard_schema.sql`. Je ne l'ai pas exécuté sur Oracle (je n'ai pas de base ici) : j'ai vérifié les décomptes par rapport au dump, l'absence de caractères parasites et l'équilibre des commentaires.

Ce qu'il contient

- 1 347 séquences
- 516 tables, avec leurs valeurs par défaut
- 266 index
- 496 clés primaires et contraintes d'unicité
- 518 clés étrangères
- 4 contraintes `CHECK`, et 328 lignes d'activation de contraintes
- 1 309 vues
- 422 fonctions et procédures
- 3 triggers sur tables
- un appel final de recompilation (`DBMS_UTILITY.COMPILE_SCHEMA`)

Les instructions sont dans l'ordre qui évite les erreurs de dépendance : séquences, tables, index, clés, contraintes, vues, puis le code PL/SQL. Les vues sont créées en mode `FORCE`, donc leur ordre n'a pas d'importance.

Ce que j'ai modifié ou retiré

- **Les clauses physiques** (`STORAGE`, `TABLESPACE`, `PCTFREE`…) sont retirées, ainsi que le préfixe de schéma `ARBIOCHEMSTANDARD`. Le script se lance donc dans n'importe quel schéma cible.

- **Les 2 triggers de connexion de base** (`AFTER LOGON ON DATABASE` , liés à l'utilisateur `SIGI`) sont en commentaire : ils demandent des droits DBA.
- **8 fonctions et procédures sont en commentaire** : leur code est tronqué dans l'export, ou contient des instructions étrangères (par exemple des `INSERT` collés après le `END;`). Ce sont `DELETEHSFABRICATION` , `DELETERESSOURCEFABRICATION` , `GETFIRSTDATEOFWEEK` , `GETSEQUNITEMAINTENANCE` , `GET_SEQBILANSECTION` , `GET_SEQXC` , `GET_SEQ_LOG_SERVICE` et `GET_SEQ_PAIE_EDITION` . Elles étaient probablement invalides dans la base d'origine. Il faudra les corriger avant de les recréer.

Ce qui n'y est pas

- **Les données** : le dump en contient (22 Mo), mais le script ne crée que la structure. Je peux décoder des tables précises et en faire des `INSERT` , par exemple les référentiels comme `ROLES` ou `DIRECTION` .
- **Les utilisateurs et les droits d'accès** (`GRANT`) : il n'y en a pas dans cet export.

Deux points à savoir

- Les valeurs de départ des séquences (`START WITH`) sont celles du moment de l'export. Sur une base neuve, tu voudras peut-être les remettre à 1.
- Les contraintes sans nom reçoivent un nom généré par Oracle à la création, donc les noms ne seront pas identiques à ceux de la base d'origine.

Si tu veux, on peut reprendre le cours : la page suivante est `PageConsulteMultiple` (la fiche d'une mère avec son tableau de filles), avec son quiz.

je voudrais que tu modifies la structure du fichier, mets les views ensemble, les tables,...
mets le script de creation des tables en premier ou ce qui doit aller en premier
et fait attention de bien mettre les bonnes tables dans l;ordre

Choix de l'ordre des vues et routines ›
Je restructure le script : tables en premier, rangées selon leurs clés étrangères (parents avant enfants), puis chaque type d'objet dans sa propre section.

Ran 3 commands, shared files ›
J'ai restructuré `arbiochemstandard_schema.sql` , avec les tables en premier. Je ne l'ai pas exécuté sur Oracle : j'ai vérifié l'ordre et la structure par analyse du fichier.

Nouvelle structure (rappelée dans l'en-tête du fichier)

1. **Tables**, avec leurs valeurs par défaut
2. **Séquences**, par ordre alphabétique
3. **Index**
4. **Clés primaires et contraintes d'unicité**
5. **Clés étrangères**
6. **Contraintes** `CHECK` et activation des contraintes
7. **Fonctions**
8. **Vues**
9. **Procédures**, suivies des 8 routines en commentaire à la source incomplète
10. **Triggers**
11. **Recompilation** des objets

L'ordre des tables

Les tables sont triées d'après leurs clés étrangères : une table parente est toujours créée avant ses tables enfants. Chaque table est numérotée (par exemple `[4/516] AGENCE`), et un commentaire indique de quelles tables elle dépend. J'ai vérifié les 518 clés étrangères : aucune ne pointe vers une table créée après la sienne. Il n'y a **aucune dépendance circulaire**.

Les clés étrangères restent dans une section séparée (la 5), après les clés primaires. Une clé étrangère a besoin de la clé primaire de sa table parente, qui est créée en section 4. Cet ordre évite donc les erreurs, et les sections 3 à 6 suivent le même ordre de tables que la section 1.

Les autres ordres

- **Les vues** sont triées par dépendance : une vue qui en utilise une autre vient après elle. Il n'y a pas de cycle, et elles restent créées en mode `FORCE`.
- **Les fonctions** passent avant les vues et les procédures, parce qu'une procédure en appelle une.

Une correction sur mes chiffres

J'avais écrit 422 fonctions et procédures actives. Le compte exact est **424 actives** (418 fonctions et 6 procédures), plus les **8 mises en commentaire**, soit 432 au total, comme dans le dump.

Le reste du contenu est inchangé (les triggers de connexion en commentaire, les clauses physiques retirées, aucune donnée). Si tu veux aussi les `INSERT` de quelques tables, par exemple les référentiels, dis-moi lesquelles, ou on reprend avec `PageConsulteMultiple`.

continuons l'apprentissage

Ran 3 commands >

Voici `PageConsulteMultiple`. J'ai lu son code (une page de 450 lignes) et j'ai constaté plusieurs différences avec `PageConsulte` qui ne sont pas dans la doc.

PageConsulteMultiple : la fiche d'une mère avec ses lignes, en détail

1. Son rôle

Elle affiche en lecture seule **deux choses l'une sous l'autre** : la fiche de la mère (comme `PageConsulte`), puis un **tableau de toutes ses filles**. C'est la page naturelle pour consulter un document créé avec `PageInsertMultiple` : le carton et ses lignes, par exemple.

2. Le constructeur : cinq éléments

1. la **mère** (l'entité, ou sa vue) ;
2. la **fille** (l'entité des lignes) ;
3. le **nom de la colonne mère** : l'attribut de la fille qui contient l'id de la mère (c'est la liaison de l'étape 2, par exemple `idMere`) ;
4. la requête ;
5. l'utilisateur.

Il enchaîne :

1. il lit l'id de la mère dans l'URL (le paramètre porte le nom de l'attribut clé primaire) ;
2. il **charge la mère** avec `UserEJB.getData` ;
3. il **charge les filles** avec une seconde recherche, qui ajoute la clause libre de l'étape 3 : « and *colonneMere* = '*id de la mère*' » ;
4. si la mère n'existe pas, il lève « Pas de resultat pour votre consultation » ;
5. il construit les champs de la mère et ceux des filles.

Deux points d'attention, d'après le code (non testés)

- **L'id oublié dans l'URL** : contrairement à `PageConsulte`, il n'y a pas d'exception « Pas d identifiant ». Le critère est alors vide, la page prendrait la première mère de la table, et aucune fille (la clause chercherait l'id « null »). C'est le même risque que dans `PageUpdate`.
- **L'id dans la clause** : la valeur de l'id lue dans l'URL est **concaténée telle quelle** dans la clause des filles, sans passer par `Utilitaire.replaceChar`. C'est exactement ce que je t'ai déconseillé pour tes propres clauses à l'étape 3. Ne copie pas ce modèle : nettoie toujours les valeurs venant de l'utilisateur.

3. La construction des champs

- **La mère** : un champ par colonne, avec la mise en forme déjà vue (valeur nulle vide, date en jour/mois/année, réels avec séparateurs de milliers). **Différence avec** `PageConsulte`

: le libellé par défaut est le **nom de la colonne**, pas l'annotation `@LibelleAffichage`. Il faut donc remplacer les libellés toi-même.

- **Les filles** : un tableau à deux dimensions, une ligne par fille et une colonne par colonne de la fille, avec la même mise en forme. S'il n'y a aucune fille, la page crée tout de même une ligne de champs vides, pour pouvoir afficher les en-têtes.

4. Ce que tu règles

- **Pour la mère** : `getChampByName` (libellé, visibilité, lien), `setLibAffichage` (tous les libellés d'un coup, dans l'ordre des champs), `setTitre`.
- **Pour les filles** : `getChampFilleByName` et `setLibAffichagefille`. Ces deux méthodes ne touchent que la **première ligne** du tableau de champs, qui sert d'en-tête. La visibilité d'une colonne, elle, est lue sur cette première ligne pour **toutes** les lignes : masquer une colonne dans l'en-tête la masque donc partout.
- `setBute` : la page vers laquelle revenir après une suppression de ligne.
- **Fichiers joints** : une option d'upload (activée par défaut), avec le nom de la table cible et de la procédure de séquence.

Un piège sur les liens des filles : pour chaque cellule, la page regarde le lien du champ **de sa propre ligne**. Le champ qu'on obtient avec `getChampFilleByName` est celui de la première ligne seulement. Pour qu'une colonne soit un lien sur toutes les lignes, il faudrait donc parcourir le tableau de champs de chaque ligne. C'est ma lecture du code, à vérifier dans ton projet.

5. L'affichage (`makeHtml`)

- **La mère** : comme dans `PageConsulte`. Moins de 13 champs, un seul tableau centré ; à partir de 13, deux colonnes. Les champs invisibles sont ignorés, et le champ `etat` est affiché sous forme de texte si la mère est un `ClassEtat`. Le titre (avec une flèche de retour) comporte l'id de la mère.
- **Le tableau des filles** : une ligne d'en-têtes, puis **une ligne par fille**. Si la fille est un `ClassEtat`, sa colonne `etat` est **masquée** automatiquement. La **dernière cellule** de chaque ligne est une petite croix qui mène vers la page `apresInsertMultiple.jsp` avec `acte=deleteMultiple`, l'id de la fille, sa classe, l'id de la mère et la page de retour (`bute`).

La croix est affichée **sans condition d'état**. Le refus de supprimer une ligne d'une mère déjà validée est donc fait côté serveur (étape 2), pas par l'affichage.

6. Ce que la page ne fait pas

Elle ne construit pas les boutons Modifier, Supprimer ou Viser : comme pour `PageConsulte`, c'est le JSP qui les place, en testant l'état de la mère (inférieur à 11). Elle a seulement un bas de page avec le bouton d'upload de fichiers.

Je prépare le questionnaire de cette page.

Voici le questionnaire sur `PageConsulteMultiple`, 5 questions dont la difficulté monte. Il couvre :

- le rôle de la page ;
- la colonne mère ;
- les libellés par défaut ;
- la croix de suppression ;
- la clause de recherche des filles, avec son risque d'injection.

Réponds dans la carte et dis-moi ce qui t'a posé problème. Si tout passe, on termine les pages avec `PageRechercheGroupe`, puis on attaque l'étape suivante : les contrôleurs `apres*.jsp` et leurs actes.

continue

Ran 5 commands >

Continue

Vérifiant ce qui est affiché dans les résultats groupés ›
Voici `PageRechercheGroupe`. J'ai lu son code (1 400 lignes) en plus de la doc.

PageRechercheGroupe : la liste groupée avec sommes, en détail

1. Son rôle

Là où `PageRecherche` liste des lignes, cette page **regroupe** les lignes par les colonnes que l'utilisateur choisit (un `GROUP BY`) et **somme** des colonnes numériques pour chaque groupe. C'est un tableau de synthèse : par exemple, le total du montant des ventes par client. Dans l'exemple de la doc (étape 3), on interroge une vue de ventes avec `idClientLib` en regroupement et `montanttotal` en somme.

Elle **n'hérite pas de** `PageRecherche` : c'est une classe séparée, qui hérite de `Page` et reprend une logique voisine (critères, intervalles, paramètre `premier`, mots-clés, pagination).

2. Le constructeur

Il ressemble à celui de `PageRecherche`, avec des éléments en plus :

- l'**entité**, de préférence une **vue** (étape 2) qui contient les colonnes de regroupement (libellés inclus) et les colonnes numériques à sommer ;
- la requête, la liste des critères, la liste des intervalles et le nombre par rangée (étape 6) ;
- les **colonnes de regroupement par défaut** ;
- les **colonnes à sommer par défaut** ;
- facultatif : une liste de colonnes pour lesquelles calculer des **pourcentages** ;
- le **nombre de listes de choix de colonnes de regroupement**, puis le **nombre de listes de choix de sommes**.

Le constructeur met l'entité et l'objet critère en **mode « groupe »**, construit le formulaire de critères et lui ajoute les listes de choix de regroupement et de somme.

3. Ce que l'utilisateur choisit

L'utilisateur règle ses regroupements et ses sommes avec des listes déroulantes. La page lit dans la requête tous les paramètres dont le nom commence par `colGroupe`, `ligneGroupe` ou `colSomme`, suivis d'un numéro (`colGroupe1`, `colSomme2`…). Elle les nettoie :

- une valeur **vide** ou égale à « - » est ignorée ;
- une colonne choisie **deux fois** n'est gardée qu'une fois ;
- si **aucune somme valide** n'est choisie, la page revient aux **colonnes à sommer par défaut** données au constructeur.

Les valeurs par défaut servent aussi à **pré-sélectionner** les listes à l'écran.

Les « lignes » de regroupement : la page distingue des colonnes (`colGroupe`) et des lignes (`ligneGroupe`) de regroupement, ce qui laisse penser à un affichage croisé. Je n'ai pas suivi ce mécanisme en détail.

4. Une vérification utile

Avant de chercher, la page contrôle que **chaque colonne à sommer est numérique**. Sinon, elle lève l'exception « Erreur durant la recherche groupée : Impossible de sommer le champ non numérique '…' ».

5. `creerObjetPage` : ce qui se passe

1. Elle lit les valeurs du formulaire et prépare les données.
2. Elle interroge la base **par le service** : l'appel est `getDataPageGroupe` de `UserEJB` (étape 4), avec le critère, les colonnes de regroupement et de somme, les intervalles, le numéro de page, la clause libre (avec les mots-clés), les colonnes pour le récapitulatif, l'ordre et le nombre de lignes par page.
3. Si `premier=true`, **aucune requête** n'est faite : le résultat est vide, comme pour `PageRecherche`.
4. Elle fabrique le **tableau récapitulatif** : une ligne « Total », le nombre de lignes, puis une colonne « Somme de … » par colonne sommée. Dans cette page, le code lit le nombre à la **dernière** position du tableau de résultats et les sommes avant. C'est une disposition différente de celle de `rechercherPage` vue à l'étape 3.
5. Elle fabrique le **tableau principal**, dont les en-têtes sont **les colonnes de regroupement, puis les colonnes de somme**, et la pagination.

Il existe une variante `creerObjetPagePourc`, qui ajoute une colonne `nombrepargroupe` et des colonnes de pourcentage. Je n'ai pas détaillé son calcul.

6. Ce que la page contient en plus

Dans le code, on trouve aussi : le glisser-déposer des colonnes, des « rubriques d'analyse », un mode graphique, une fenêtre modale pour enregistrer une analyse, et une version « multiple ». La doc n'en parle pas, donc je ne les détaille pas ici.

7. L'affichage

La méthode `makeHtml` de cette classe est **vide** : c'est ton JSP qui appelle chaque élément (formulaire, tableau récapitulatif, tableau principal, pagination) et les place où il veut.

Je prépare le questionnaire de cette page.

Voici le questionnaire sur `PageRechercheGroupe`, 5 questions dont la difficulté monte. Il couvre :

- le rôle de la page ;
- les colonnes du tableau ;
- le nettoyage des choix de l'utilisateur ;
- le contrôle des colonnes à sommer ;
- le retour aux valeurs par défaut.

Réponds dans la carte et dis-moi ce qui t'a posé problème. Il reste une page à voir, `PageRechercheChoix` (la pop-up de sélection). Ensuite, on passe aux contrôleurs `apres*.jsp` et à leurs actes, qui complètent la boucle vue ici : la page construit le formulaire, la page `apres` l'enregistre.

continue

Ran 5 commands ›
Je termine la lecture de `PageRechercheChoix` (la pop-up de sélection) pour te l'expliquer, avec un point à vérifier sur le libellé renvoyé.

Ran 2 commands ›
PageRechercheChoix : la pop-up de sélection, en détail

J'ai lu son code (470 lignes), celui de `Champ` et la partie du tableau de résultats qui sert à choisir une ligne.

1. Son rôle

C'est la page qui s'ouvre quand l'utilisateur clique sur la **loupe** d'un champ de formulaire, par exemple le champ « client » d'une saisie. Elle affiche une liste filtrable de clients. L'utilisateur coche une ligne, et sa valeur est **renvoyée dans le champ** du formulaire d'où il vient.

Du côté du formulaire d'origine, on la déclare avec `setPageAppel` (étape 5) : on donne la page de choix (par exemple `choix/client-choix.jsp`) et le champ qui recevra la valeur. La

page de choix est une page JSP qui crée une `PageRechercheChoix`. Comme `PageRechercheGroupe`, elle **hérite de** `Page`, pas de `PageRecherche`.

2. Les constructeurs

Les quatre versions ont en commun l'entité, la requête, la liste des critères, celle des intervalles et le nombre par rangée. Les différences :

- la version **minimale**, sans rien d'autre ;
- une version avec le `champReturn` (le champ qui recevra la valeur) ;
- une version avec les **colonnes affichées par défaut** et leur nombre (les sélecteurs de colonnes de l'étape 6) ;
- la même, avec en plus l'indicateur `premier`.

Différence avec `PageRecherche` : ici, le « nombre par rangée » est **bien transmis** au formulaire dans toutes les versions. C'est la version à 7 paramètres de `PageRecherche` qui l'ignorait.

3. Comment les critères sont lus

La construction du critère (`makeCritere`) suit le principe de `PageRecherche` :

- un **texte** est posé sur l'objet critère (recherche « contient », ou égalité si le champ est en recherche précise) ;
- un **intervalle** lit `nom1` et `nom2` ;
- un **nombre ou une date** qui n'est pas un intervalle devient une condition `like` (ou `=`) dans la clause libre ;
- le **tri** se fait avec les mêmes paramètres, et un `order by` est ajouté à la fin.

Un point d'attention : pour les nombres et les dates, la valeur postée est **concaténée telle quelle** dans la clause. C'est le même risque d'injection que celui de l'étape 3. Ne reproduis pas ce schéma dans tes propres clauses sans `Utilitaire.replaceChar`.

`setPreciserChoix` : quand il est activé (et que ce n'est pas le premier affichage), l'objet critère passe en **mode « choix »**. Ce que ça change dépend de ton entité, je n'ai pas vu ce mode utilisé dans `apj-core`.

4. Ce que fait `creerObjetPage`

C'est la même idée que pour `PageRecherche`. Il existe plusieurs versions : avec les en-têtes seuls, avec en plus la liste des colonnes à sommer, ou avec une liste déjà chargée. L'appel :

1. lit les valeurs du formulaire ;
2. ouvre sa propre connexion et lance la recherche **via** `UserEJB` (étape 4) ;
3. charge les listes du formulaire de critères et mémorise le critère en session ;
4. prépare le tableau récapitulatif ;
5. construit le tableau des résultats.

La recherche dépend du paramètre `recap` : s'il vaut `checked`, la page calcule le récapitulatif (`getDataPageMax`), sinon elle l'évite (`getDataPageMaxSansRecap`), ce qui est plus léger.

Contrairement à `PageRecherche`, `preparerDataList` ne consulte **pas** le paramètre `premier`. Si tu veux un premier affichage sans requête, il faudra donc gérer ce cas dans ton JSP.

5. La pagination

`getBasPage` affiche « Nombre de résultat », « page X sur Y » et les liens précédent et suivant. Ces liens **recopient** `champReturn`, ce qui permet à la pop-up de savoir où renvoyer la valeur d'une page à l'autre. La méthode voisine `getBasPagee`, elle, a la valeur `etudiant` écrite en dur dans ses liens. C'est un reste d'un ancien projet : utilise `getBasPage`.

6. Le choix d'une ligne et le retour de la valeur

Le tableau des résultats se rend en **boutons radio** (méthode `getHtmlWithRadioButton` du `TableauRecherche`, avec des variantes). Chaque radio porte :

- le nom `choix` ;
- une valeur de la forme `identifiant;libellé`, par exemple `CLI000012;Rakoto` ;
- un appel à une fonction JavaScript `getChoix()` quand on clique.

Il existe aussi des variantes en **cases à cocher** pour un choix multiple.

Du côté du champ qui a ouvert la pop-up, le `champReturn` est de la forme `nomChamp;nomChamplibelle`, c'est-à-dire deux éléments : le champ qui reçoit l'**identifiant** (caché) et le champ qui reçoit le **libellé** (affiché). Le JavaScript `getChoix()` n'est pas dans `apj-core` : d'après la structure, il lit la valeur du radio, la coupe au point-virgule, et remplit les deux champs. Je ne l'ai pas vérifié.

D'où vient le libellé ? La partie après le point-virgule vient de la méthode `getValColLibelle` de ton entité. Dans `ClassMAPTable`, elle renvoie un attribut qui vaut `null` par défaut : le libellé est alors **l'identifiant lui-même**. Les entités de type `TypeObjet` la redéfinissent pour renvoyer la colonne `val`. Pour qu'une pop-up de choix affiche un nom lisible, il faut donc que ton entité de recherche (typiquement une vue `Lib`) renseigne ce libellé.

7. Ce que tu retiens

La boucle est : le champ du formulaire ouvre la page de choix avec son `champReturn`, la page cherche et affiche des radios `id;libellé`, et un script remet la valeur dans le formulaire d'origine. Rien n'est enregistré par cette page.

Je prépare le questionnaire de cette page.

Voici le questionnaire sur `PageRechercheChoix`, 5 questions dont la difficulté monte. Il couvre :

- le rôle de la pop-up ;
- le choix d'une ligne et le retour de la valeur ;
- le libellé renvoyé ;
- le paramètre `champReturn` dans la pagination ;
- la gestion de `premier`, différente de celle de `PageRecherche`.

Réponds dans la carte et dis-moi ce qui t'a posé problème. Avec cette page, on a vu toutes les pages de la couche d'affichage. L'étape suivante est celle des contrôleurs `apres*.jsp` et de leurs actes : c'est eux qui reçoivent les formulaires construits par ces pages et qui appellent `UserEJB`.

continue

Ran 2 commands ›

Étape 7 : les contrôleurs `apres*.jsp` et leurs actes (partie 1)

Cette partie boucle ce que tu as vu aux étapes 5 et 6 : les pages construisent un formulaire, la page `apres` le reçoit et enregistre. Les fichiers `apres*.jsp` appartiennent à **ton projet**, pas à `apj-core`, donc je m'appuie sur la doc. Si tu veux, tu pourras me donner le lien GitHub plus tard pour comparer avec un vrai fichier.

1. Le principe

Un `apres*.jsp` est un **contrôleur sans vue**. Il n'affiche rien : il lit le paramètre `acte`, reconstruit les entités depuis la requête, appelle `UserEJB`, puis **redirige** vers une autre page avec un petit script JavaScript. On y arrive par l'adresse `module.jsp?but=apresMultiple.jsp`, ou `apresTarif.jsp` pour les entités simples (c'est la convention du projet d'exemple de la doc).

2. Le contrat des paramètres

Les formulaires et les liens d'action transmettent ces paramètres :

Paramètre	Rôle
`acte`	l'action à exécuter (`insert`, `update`, `delete`, `valider`…)
`classe`	le **nom complet de la classe** de l'entité, instanciée par `Class.forName`
`classefille`	la classe des filles (actes multiples)
`bute`	la page de destination **après succès** (par défaut `pub/Pub.jsp`)
`id` / `ids`	l'identifiant d'un objet, ou les identifiants cochés
`nomtable`	force la table ou la vue de l'entité (`setNomTable`, étape 2)
`colonneMere`, `nombreLigne`	pour mère et filles : la colonne de liaison et le nombre de lignes
`rajoutLien`	des paramètres à recopier sur l'URL de redirection
`type`	le type de bouton (3 par défaut, qui veut dire modifier)

3. Le déroulé de l'acte `insert` (mère et filles)

Voici ce que fait `apresMultiple.jsp`, étape par étape :

1. Il **instancie** la mère et la fille avec les noms de classes reçus.
2. Il crée une `PageInsertMultiple` avec ces objets, la requête, le nombre de lignes et la liste des lignes cochées.
3. Il reconstruit la **mère** avec `getObjectAvecValeur`, ce qui déclenche les contrôles des setters (étape 1).
4. Il reconstruit les **filles** avec `getObjectFilleAvecValeur`, **cochées seulement** (étape 6).
5. Si `nomtable` est fourni, il le pose sur chaque fille.
6. Il appelle `u.createObjectMultiple(mère, colonneMere, filles)`, qui renvoie la mère enregistrée, avec son id.
7. Il **redirige** vers `bute` en ajoutant `&id=` suivi de l'id de la mère.

4. Ce qui se passe ensuite dans le service (rappel des étapes 1, 2 et 4)

`createObjectMultiple` contrôle le droit `insert` (`ActionRole`), ouvre la connexion avec l'auto-commit désactivé, renseigne l'état à 1 et l'auteur, puis rattache les filles à la mère. L'entité `ClassMere` enchaîne alors :

1. `controler` , puis `construirePK` de la mère (par exemple `CRT000042`) ;
2. l'insertion avec historique ;
3. pour chaque fille, la colonne de liaison reçoit l'id de la mère, puis `controler` , `construirePK` (par exemple `CRTF…`) et l'insertion en lot.

Le tout est dans **une seule transaction**.

5. Deux variantes proches

- **Si le paramètre de clé primaire est renseigné**, l'acte `insert` devient automatiquement une mise à jour (`updateInsert`). C'est ce que tu as vu avec le champ `id` en lecture seule de `PageUpdate` .
- **`insertSansControle`** : le même acte, avec la version de la reconstruction des filles qui **ne lève pas d'exception si aucune ligne n'est cochée** (étape 6).

6. La gestion des erreurs

Toute exception est attrapée par le contrôleur, qui affiche une alerte avec le message puis revient en arrière. Le message que verra l'utilisateur est donc celui que tu écris dans ton `controler()` ou dans tes setters. Comme la transaction a été annulée dans le service, **rien n'a été enregistré**.

7. Trois points d'attention (donnés par la doc)

- Les JSP déclarent des **variables d'instance** (`<%! UserEJB u; String acte; %>`). Comme un JSP est une servlet partagée entre les requêtes, deux utilisateurs peuvent se marcher dessus. Dans un nouveau contrôleur, utilise des variables locales.
- La classe est instanciée depuis un **paramètre HTTP**. Il faut la limiter aux classes `ClassMAPTable` . Les droits sont, eux, vérifiés par `ActionRole` dans l'EJB.
- Le projet expose aussi une servlet `/ApresMultiple` pour des actes appelés en Ajax, qui répond en JSON avec un statut `success` ou `error` .

8. Écrire son propre contrôleur

C'est le même schéma : récupérer l'utilisateur `u` en session et l'acte, tester l'acte, faire l'opération métier avec `u` , rediriger vers la page voulue, et attraper l'exception pour afficher son message.

Je prépare le questionnaire de cette partie. Dans la partie 2, on verra le catalogue des autres actes : `update` , `delete` , les actes de workflow, `dupliquer` et les actes métier.

Voici le questionnaire sur les contrôleurs `apres*.jsp` (partie 1), 5 questions dont la difficulté monte. Il couvre :

- le rôle d'un contrôleur ;
- le paramètre `bute` ;
- le passage de `insert` à `update` ;
- l'ordre des opérations de l'acte `insert` ;
- la gestion d'une exception.

Réponds dans la carte et dis-moi ce qui t'a posé problème. Ensuite, la partie 2 montrera le catalogue des autres actes : `update`, `delete`, les actes de workflow (`valider`, `annuler`, `cloturer`…), `dupliquer`, et comment écrire un acte métier à toi.

mais comment savoir quel apres(...).jsp utilise?

Checking which apres page each project uses ›

Le choix du fichier `apres` n'est **pas automatique** : c'est toi, le développeur, qui l'écris dans l'adresse du formulaire ou du lien. Le framework ne le choisit pas.

1. Où ce choix est écrit

- **Pour un formulaire** (saisie ou modification) : dans l'attribut `action` de la balise `<form>`, par exemple `module.jsp?but=apresTarif.jsp` ou `?but=apresMultiple.jsp`.

- **Pour un lien d'action** dans une fiche (Supprimer, Viser) : dans l'adresse du lien, par exemple `?but=apresTarif.jsp&acte=delete&id=…`.

Le paramètre `acte` dit ensuite **quoi faire.** Le fichier `apres` est donc le « où », et l'acte est le « quoi ».

2. La règle suivie par le projet d'exemple de la doc

Situation	Fichier utilisé
Entité simple (`PageInsert` , `PageUpdate`)	`apresTarif.jsp`
Actions d'une fiche simple (supprimer, viser)	`apresTarif.jsp`
Mère et filles (`PageInsertMultiple` , modification multiple)	`apresMultiple.jsp`

Ces noms sont une convention **de ce projet**. Le framework ne l'impose pas.

3. Un exemple qui montre que les noms varient

Dans le code de `PageConsulteMultiple` (étape 6), la croix de suppression d'une fille pointe vers un fichier nommé **`apresInsertMultiple.jsp`** , alors que la doc parle de `apresMultiple.jsp` . Le nom est écrit en dur dans `apj-core` , et peut donc ne pas correspondre au nom de ton projet. Dans ce cas, il faut soit un fichier de ce nom, soit adapter le lien.

4. Comment savoir dans ton projet

1. **Regarde une page qui fait déjà la même chose.** Ouvre le JSP de saisie d'une entité comparable à la tienne, et lis l' `action` de son formulaire : c'est le fichier à réutiliser.
2. **Ouvre le fichier `apres` et cherche les actes qu'il gère.** Il contient une suite de tests sur `acte` (`insert` , `update` , `delete` , `valider` …). Si ton acte y figure, c'est le bon fichier.
3. **Si l'acte n'existe nulle part**, tu écris ton propre contrôleur ou tu ajoutes un test à un fichier existant, comme dans l'exemple `livrer` de la doc.

5. Une aide possible

Si tu m'envoies le lien GitHub du projet, je peux lire ses vrais fichiers `apres*.jsp` et te dire lesquels existent, quels actes chacun gère et lequel utilise chaque page de saisie. Sinon, je continue avec la doc.

Est-ce plus clair ? Si oui, tu peux répondre au quiz de la partie 1, puis on passe à la partie 2 (le catalogue des autres actes).

continue

Ran 5 commands ›

Je lis les classes d'état, de mère et de fille dans `apj-core` pour vérifier chaque acte du catalogue, et j'ai trouvé deux écarts avec la doc que je te signale plus bas.

Étape 7 : les contrôleurs `apres*.jsp` et leurs actes (partie 2)

Rappel : le fichier `apres` dit **où** on va, et l'acte dit **quoi** faire. Voici les familles d'actes du catalogue de la doc.

1. Modifier

- **`update`** : le contrôleur reconstruit l'objet avec `PageUpdate.getObjectAvecValeur`, puis appelle `u.updateObject(objet)`. Pour un objet avec état (`ClassEtat`), la méthode refuse si l'état est supérieur ou égal à 11 : « Impossible de modifier un objet deja valide ». Sinon, elle met à jour la ligne et écrit l'historique.
- **`updateInsert`** et **`updateSansControle`** : modification d'une mère et de ses filles, avec `u.updateObjectMultiple(mère, colonneMere, filles)`. Redirection vers `bute&id=…`.
- **`updateMultiple`** : modification de plusieurs objets d'un tableau, avec `u.updateObjectMultiple(objets)`.

2. Supprimer

- **`delete`** : le contrôleur pose l'id sur un objet vide, puis appelle `u.deleteObject`. En cas d'erreur : alerte et retour en arrière.
- **`deleteMultiple`** et **`deleteFille`** : suppression de plusieurs objets, ou d'une fille. C'est ce que déclenche la croix de `PageConsulteMultiple` (étape 6). D'après la doc, la suppression d'une fille est refusée si la mère est déjà validée.

3. Le workflow (les états de l'étape 2)

- **`valider`** : `u.validerObject`. Dans `ClassEtat`, la méthode passe en mode modification, lance `controlerUpdate`, envoie une notification si un message est prévu, puis met l'état à **11** si l'objet est à l'état 1 à 10 et que sa validation n'est pas hiérarchique. Un objet annulé (état 0) n'est donc pas passé à 11. Si une règle de visa hiérarchique existe pour la classe (la table `RoleValidation`), l'état n'est pas fixé directement : c'est la méthode `viser` qui gère la chaîne des visas.
- **`attacher`** (l'acte par défaut des tableaux à cases à cocher) : elle boucle sur `ids[i]` et valide chaque objet. Il existe aussi **`validerMultiple`** et **`validerTous`**.
- **`cloturer`** : l'état passe à **9**, après un appel à `controler`.
- **`annuler`** : l'état passe à **0**.
- **`annulerVisa`** : l'état repasse à **1**.
- **`finaliser`** : une étape métier propre au projet.

4. Dupliquer

L'acte `dupliquer` appelle `u.dupliquerObject(objet, classeFille, colonneMere)`. Dans `apj-core`, l'entité de base crée un nouvel objet **du même type**, en **copiant tous ses**

attributs par réflexion, puis fabrique une nouvelle clé avec `construirePK`. Pour une mère, `ClassMere` va plus loin :

1. elle met l'état de la copie à 1 et l'insère avec son historique ;
2. elle charge les filles si ce n'est pas fait ;
3. elle duplique chaque fille, avec la colonne de liaison pointant vers la **nouvelle** mère, et les insère en lot ;
4. tout se passe dans **une seule transaction** (commit, ou rollback en cas d'erreur).

La redirection se fait vers `bute&id=nouvelId`. C'est ce qui explique le bouton « Dupliquer » de `PageConsulte` (étape 6).

5. Les autres actes

- **`insertFille`** : ajoute des filles à une mère déjà enregistrée (l'id de la mère est obligatoire), avec `createObjectFilleMultiple`.
- **`insertFilleSeul`** : crée chaque fille avec `u.createObject`.
- **`insertMereLierFille`** : crée une mère et la lie à des filles existantes (`insertMereLierFilles`).
- **`insertTypeObjet`** : crée plusieurs `TypeObjet` d'un coup (champs `val_i` et `desce_i`).
- **Les actes métier** (`viserTraite`, `debaucher`, `interview`…) : propres au projet, dans l'esprit de l'exemple `livrer` de la doc.

6. Deux écarts à connaître entre la doc et `apj-core`

- **La validation de la mère sur ses filles.** `ClassMere.validerObject` appelle bien `validerObject` de chaque fille avant la sienne. Mais, dans `apj-core`, la méthode de base de `ClassFille` **ne fait rien** (elle renvoie `null`). La propagation n'a donc un effet **que si ta classe fille redéfinit `validerObject`**. Je t'avais dit à l'étape 2 que la validation se propage aux filles : c'est vrai de la mère, mais l'effet sur les filles dépend de tes classes.
- **Deux méthodes de contrôle presque homonymes.** `ClassEtat` a `controleAnnule` (sans R), un hook vide appelé par `annulerObject`, et `controleAnnuler` (avec un R), qui lève « Impossible à annuler, deja valide » si l'état est au moins 11 et qui est appelée par une autre méthode, `annuler`. Même piège pour le visa : `controlerAnnulationVisa` (vide, appelée par `annulerVisa`) et `controleAnnulerVisa` (qui refuse si un supérieur a validé). Je ne vois pas, dans `apj-core`, quelle méthode ton `UserEJB` appelle pour l'acte `annuler` : la doc dit qu'un objet validé est refusé, mais ça dépend de ce choix. Si tu redéfinis un contrôle dans ton entité, vérifie le nom exact.

Je prépare le questionnaire de cette partie.

Voici le questionnaire sur les actes des contrôleurs `apres*.jsp` (partie 2), 5 questions dont la difficulté monte. Il couvre :

- la modification d'un objet déjà validé ;
- la validation et ses conditions ;
- l'acte `attacher` ;
- la duplication d'une mère et de ses filles ;
- la propagation de la validation aux filles.

Réponds dans la carte et dis-moi ce qui t'a posé problème. Si tout passe, on enchaîne sur l'étape suivante : les servlets, les plugins JS et les conventions HTTP (`module.jsp` , l'autocomplétion, les listes déroulantes dépendantes), puis le flux complet du module « Carton » qui relie toutes les étapes.

peux-tu me creer a partir du .dmp un script vierge contenant uniquement les tables,sequences,vues,fonctions,triggers, de base que l'on doit avoir pour n'importe quel projet utilisant le framework avec les donnees necessaires, classe le script pour separer table,vue,seq,fonction,.. mais dans l'ordre d'execution requise

Recensement des objets du framework à partir du dump ›

Je commence par identifier, dans `apj-core` et dans la doc, quelles tables, séquences, vues et fonctions le framework lui-même utilise. Je les croise ensuite avec le dump pour séparer

le socle du métier propre au projet.

Ran 6 commands >

Je construis le script de base à partir du dump. Je vérifie maintenant les vues, les fonctions, les séquences et les triggers qui servent au socle.

Ran 16 commands, shared files >

Paused when your free usage ran out. Continue to pick up where Claude stopped. Continue